Future Generation Computer Systems 182 (2026) 108492

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

Generative artificial intelligence-driven secure pseudorandom number generator

Xuguang Wu [a,b], Yiliang Han [a,b,*], Minqing Zhang [a,b], Shuaishuai Zhu [a,b], Xu An Wang [a,b]

[a] College of Cryptography Engineering, Engineering University of People's Armed Police, Xi'an, 710086, Shaanxi, China
[b] Key Laboratory of Network and Information Security under the People's Armed Police, Xi'an, 710086, Shaanxi, China

ARTICLE INFO

Keywords:
Generative adversarial networks (GANs)
Pseudorandom number generators (PRNGs)
Learning parity with noise (LPN) problem

ABSTRACT

The rapid progress of generative artificial intelligence (GenAI), especially generative adversarial networks (GANs), has stimulated growing interest in learning-based pseudorandom number generation. However, most existing GAN-based PRNGs are evaluated primarily through empirical statistical batteries, providing limited formal assurance against computationally powerful adversaries and lacking reductions to established hardness assumptions. This paper proposes a PRNG construction that integrates a GAN-based Bernoulli noise expansion sampler into an LPN-based framework. We formalize a neural-structured LPN variant (GAN-LPN) that captures practical non-idealities of learned sampling via explicit, measurable deviation parameters, and we carry these terms through the security analysis as additive degradation in the final advantage bound. We then present a game-based reduction showing that, under the stated assumptions and deviation bounds, distinguishing the PRNG output from random can be reduced to the hardness of the corresponding underlying LPN-type problem. Experimental results show that the generated sequences pass NIST SP 800-22 and additional empirical benchmarks, including the Hamming Distance test and a practical next-bit prediction test, providing supplementary evidence on the statistical quality of the output.

1. Introduction

The rapid evolution of Generative Artificial Intelligence (GenAI) [1,2] has attracted increasing attention in cybersecurity research. At the cryptographic foundation of secure systems, GenAI may offer new possibilities for the design of traditional cryptographic primitives, particularly Pseudorandom Number Generators (PRNGs) [3,4]. As a cornerstone of critical security mechanisms, including key generation, encryption protocols, and authentication frameworks, PRNGs require rigorous security guarantees as well as robust performance properties. In particular, GenAI techniques, especially Generative Adversarial Networks (GANs) [5], provide flexible tools for modeling complex distributions, which may be useful in the design of efficient PRNG components.

Recently, preliminary research has emerged exploring the use of GANs for constructing efficient PRNGs. In 2018, Bernardi et al. [6] first introduced GANs into the design of PRNGs and proposed two operational modes: discriminative mode and predictive mode. In 2019, Oak et al.[7] presented a discriminative-mode-based PRNG architecture based on a three-layer fully connected neural network. In 2021, Kim et al. [8] proposed a predictive-mode PRNG leveraging similar generative models. Subsequently, in 2023 and 2024, respectively, Okada et al. [9] and Wu et al. [10] proposed pseudorandom number generators based on variants of the Wasserstein GAN (WGAN) [11]. A potential advantage of these approaches lies in their computational efficiency, enabling the generation of random sequences with target statistical properties. However, existing GenAI-based PRNG schemes generally lack formal security guarantees. Most existing methods rely on the statistical fitting capabilities of generative models, with security evaluations typically based on empirical randomness tests rather than formal cryptographic proofs grounded in established computational hardness assumptions. Consequently, the observed statistical randomness of the output may not by itself provide strong assurance against adversaries with substantial computational power, including potential quantum adversaries, and may therefore be insufficient for the security standards required in modern cryptography.

To address these limitations, this paper proposes a framework for a secure pseudorandom number generator based on generative artificial intelligence. Our main contributions are summarized as follows.

(1) **A neural Bernoulli Noise Expansion Sampler (BNES) for LPN-style constructions.** We devise a sampler built upon a Wasserstein GAN

* Corresponding author.
E-mail addresses: wxguang0210@163.com (X. Wu), hanyil@163.com (Y. Han), api_zmq@126.com (M. Zhang), zsseupap@163.com (S. Zhu), wangxazjd@163.com (X.A. Wang).

https://doi.org/10.1016/j.future.2026.108492
Received 30 June 2025; Received in revised form 22 January 2026; Accepted 21 March 2026
Available online 28 March 2026

Table 1
Notation and parameters used throughout the paper.

Symbol	Description
$\mathbb{F}_2$	Finite field with two elements
n	Length of the LPN sample vector and noise vector
m	Internal linear state size of the PRNG ($m \le n$)
ℓ	Length of the BNES input (latent seed)
μ	Output length per iteration, $\mu = n - m - \ell$
$M \in \mathbb{F}_2^{m \times n}$	Fixed binary matrix used in the LPN-style PRNG core computation
$v_t \in \{0,1\}^m$	Linear internal state at iteration t
$r_t \in \{0,1\}^\ell$	BNES input at iteration t
$w_t = v_t \Vert r_t$	Full internal state of the PRNG at iteration t
$e_t \in \{0,1\}^n$	Noise vector generated by BNES at iteration t
$c_t \in \{0,1\}^n$	Intermediate vector $c_t = M^\top v_t \oplus e_t$
$z_t \in \{0,1\}^\mu$	Output bitstring at iteration t
seed	PRNG seed initializing $w_0 = v_0 \Vert r_0$, seed $\in \{0,1\}^{m+\ell}$
$\oplus$	Bitwise XOR operation
$\Vert$	Concatenation of bitstrings
BNES$(\cdot)$	GAN-based noise sampling function
t	Iteration index

with Gradient Penalty (WGAN-GP) [12] framework, which expands a short uniform seed into a long n-bit noise vector. The sampler is architecturally constrained through independent subnetworks and independent seeding, with the goal of promoting bit-wise statistical independence rather than relying solely on unconstrained generative learning.

(2) **GAN-LPN: an LPN instantiation with learned, bounded-rate product noise and explicit deviation terms.** We formalize a GAN-induced LPN distribution in which each noise coordinate follows a Bernoulli distribution with a possibly non-identical rate ($p_i \in [\delta_1, \delta_2]$). Any departure from an ideal product distribution is quantified by an explicit statistical-distance term (ϵ_{ind}) (and optionally by a rate-mismatch term (ϵ_{bias})). Unlike correlated-noise LPN formulations that presuppose structured noise correlations, our formulation is intended to preserve the standard LPN hardness baseline while incorporating non-idealities as explicit, measurable degradation terms in the security reduction.

(3) **A PRNG construction and evaluation.** Using the BNES as a noise source, we construct a stateful LPN-style pseudorandom number generator whose pseudorandomness is analyzed via a reduction to the hardness of the underlying LPN problem. We also provide an empirical evaluation, including statistical randomness tests, dependency analyses, a practical next-bit prediction test, adversarial distinguisher attacks with re-trained models, and performance benchmarking.

The remainder of this paper is structured as follows. Section 2 reviews the necessary preliminaries and related works. Section 3 details our core construction, which comprises: (1) a GAN-based Bernoulli Noise Expansion Sampler that serves as a noise source, (2) the formal definition of an LPN variant that incorporates the properties of our neural generator, and (3) the complete architecture and iterative procedure of the proposed PRNG. Section 4 provides the security analysis. Section 5 presents a concrete instantiation of the generator. Finally, Section 6 concludes the paper by summarizing the contributions and outlining future research directions. For clarity and consistency, Table 1 summarizes the notation and parameters used throughout the paper.

2. Preliminaries and related works

In this section, we provide the necessary background and review related work essential to the development of our proposed PRNG. This includes a brief overview of GANs, existing approaches to PRNG design, the LPN problem, and methods for distribution sampling.

2.1. Generative adversarial networks

Generative adversarial networks [5], first introduced by Ian Goodfellow et al., represent a groundbreaking machine learning paradigm for estimating high-dimensional data distributions through adversarial optimization methods. At the heart of GANs lies a min-max game between two neural networks: the Generator (G), which generates data samples from a latent space, and the Discriminator (D), designed to distinguish real data from those generated by G. While the theoretical elegance of GANs has inspired extensive research, recent comprehensive surveys[13,14] systematically document the rapid evolution of GAN architectures and their expanding applications in cybersecurity and cryptographic contexts.

While theoretically elegant, standard GANs often suffer from training instability and mode collapse. The WGAN[11] addresses these issues by adopting the Wasserstein-1 distance as the training objective, which provides more stable gradients. The WGAN with Gradient Penalty[12] further enhances training robustness by replacing weight clipping with a gradient penalty term to enforce the Lipschitz constraint. These innovations marked a significant step towards stable generative modeling, which remains a benchmark for training stability in scenarios requiring precise distribution matching[15]. Its superior stability and theoretical grounding make it a suitable foundation for our cryptographic noise sampler, ensuring reliable and high-quality generation of Bernoulli-distributed sequences.

2.2. Pseudorandom number generators

PRNGs are deterministic algorithms that form the bedrock of modern cryptographic systems, expanding a short seed into a long sequence of bits that is computationally indistinguishable from true randomness. In practical deployments, PRNGs are typically instantiated within standardized deterministic random bit generator (DRBG) frameworks. In particular, the NIST SP 800-90 series specifies widely adopted constructions and requirements, including approved DRBG designs based on block ciphers, hash functions, and HMAC (SP 800-90A)[16], entropy source requirements and health tests (SP 800-90B)[17], and recommendations for random bit generation systems combining entropy sources and DRBGs (SP 800-90C)[18]. These standards underpin many widely deployed cryptographic libraries and protocols.

Beyond standardized DRBG constructions, a variety of classical PRNG algorithms have been proposed and studied in the literature, differing in design goals, security assumptions, and performance characteristics. Representative examples include the number-theoretic Blum-Blum-Shub (BBS) generator [19], the statistically oriented Mersenne Twister (MT) [20], the stream-cipher-based HC-128 PRNG [21], and the ChaCha PRNG derived from the ChaCha stream cipher [22]. These generators are typically implemented in software, making them computationally efficient and suitable for deployment on general-purpose computing platforms, while offering different tradeoffs between provable security, statistical quality, and throughput.

With the advancement of generative artificial intelligence, its application to the design of pseudorandom number generators has recently attracted increasing attention. The exploration of GAN-based PRNGs typically frames the problem as an adversarial game: a generator network learns to produce sequences that deceive a discriminator network trained to distinguish them from genuine random bits. Initial work by Bernardi et al.[6] established two fundamental modes for this task: the discriminative mode, where the generator directly outputs random vectors, and the predictive mode, where it predicts subsequent bits. This pioneering study demonstrated that neural networks could achieve high pass rates on standard statistical test suites like NIST SP800-22. Subsequent research branched out along several paths to address core challenges. Oak et al.[7] and Kim et al.[8] explored different neural architectures, employing fully-connected networks and hybrid RNN-convolutional designs, respectively, to enhance the modeling of random sequences. A significant challenge in this field is the inherent instability of GAN training. To tackle this, Okada et al.[9] leveraged the WGAN-GP, a more stable variant, to transform non-uniform inputs into approximately uniform outputs. However, their design traded off strict PRNG structural requirements for distributional accuracy. Most

recently, Wu et al. [10] introduced an innovative hybrid approach, integrating genetic algorithms into the GAN framework to dynamically optimize the generator's parameters. This meta-optimization strategy demonstrated improved robustness and passed both NIST SP800-22 [23] and the more stringent Chinese GM/T standards [24].

Despite these progressive innovations in architecture and training, a critical limitation pervades the entire body of work: the reliance on statistical testing as the sole security validation. None of the existing schemes provide a formal security reduction to a well-established hard problem. This fundamental gap between empirical performance and provable security highlights the necessity of our work, which seeks to embed the statistical prowess of GANs within a cryptographically secure foundation.

2.3. Learning parity with noise problem

Let $\mathbb{F}$ be a finite field, $m, n \in \mathbb{N}$, a secret vector $\boldsymbol{s} \in \mathbb{F}_2^m$, and a noise rate $p \in (0, 1/2)$. The Learning Parity with Noise problem is defined as follows: given access to multiple independent samples of the form

$$(A, \mathbf{b}) \in \mathbb{F}_2^{n\times m} \times \mathbb{F}_2^n,$$

where each sample satisfies

$$\mathbf{b} = A\boldsymbol{s} + \mathbf{e},$$

with $A \sim \mathrm{Unif}(\mathbb{F}_2^{n\times m})$ and $\mathbf{e} \sim \mathrm{Ber}(p)^n$, the goal is to recover the secret vector $\boldsymbol{s} \in \mathbb{F}_2^m$.

The corresponding LPN distribution, denoted $\mathrm{LPN}_{m,n,p}(\boldsymbol{s})$, is the joint distribution of such pairs $(A, \mathbf{b})$ under the above sampling process. In this setting, each row of the matrix A can be interpreted as a random query vector $\boldsymbol{a}_i \in \mathbb{F}_2^m$, and the corresponding component $b_i = \langle \boldsymbol{a}_i, \boldsymbol{s}\rangle + e_i \bmod 2$ represents the noisy parity of $\boldsymbol{s}$ with respect to $\boldsymbol{a}_i$, corrupted by an independent Bernoulli noise bit e_i that flips the result with probability p.

The hardness of the LPN problem increases as the noise rate p approaches $1/2$, where the noise becomes indistinguishable from uniform randomness. This makes distinguishing the correct linear relation between $\boldsymbol{a}_i$ and $\boldsymbol{s}$ increasingly difficult. Over the past decade, numerous variants of the LPN problem have been researched to meet diverse cryptographic requirements, achieved by modifying the secret distribution, noise model, algebraic structure, or adversarial capabilities. These variants include: Sparse LPN[25], where the secret vector contains limited non-zero entries to improve computational efficiency; Dense-Sparse LPN[26], which can resist subexponential-time adversaries through the structured matrix distribution of "dense matrix T-sparse matrix M"; Large-Field LPN[27], extending the problem to arbitrary finite fields to support advanced cryptographic primitives; Ring-LPN[28], leveraging ring structures for efficiency gains, particularly suitable for lightweight protocols; and Low-Noise LPN[29], operating under minimal noise rates to enable public-key encryption and key security. In this work, we introduce a new variant termed GAN-LPN, which uniquely replaces the conventional static Bernoulli noise source with a generative neural network.

2.4. Data sampling methods

Data sampling, the process of generating representative samples from a target distribution, is a fundamental component in cryptographic system design. While classical sampling techniques [30] such as inverse transform sampling, rejection sampling, and Markov Chain Monte Carlo (MCMC) methods offer well-understood theoretical guarantees, they face significant limitations in cryptographic applications. These methods typically require the input dimension to be at least equal to the output dimension, imposing substantial entropy and computational overhead when generating high-dimensional vectors-a critical drawback for efficient noise generation in cryptographic primitives.

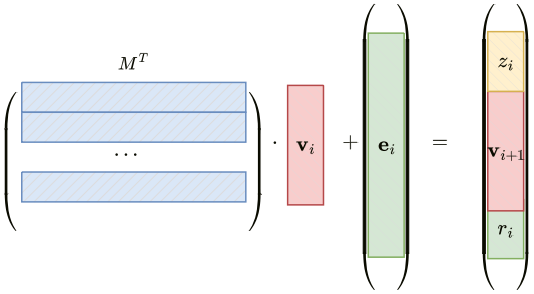

Fig. 1. PRNG construction based on the LPN problem.

In contrast, neural network-based sampling has emerged as a powerful alternative, capable of learning complex distributions and performing low-to-high-dimensional mapping. A series of theoretical advances have established the strong expressivity of neural samplers: In 2019, Horger et al. [31] demonstrated that fully connected networks can effectively map uniform samples to target density functions. In 2020, Perekrestenko et al. [32] proved that deep networks with sawtooth activations can approximate Lipschitz-continuous distributions with controlled Wasserstein error; Mukherjee et al. [33] derived explicit bounds on the neuron count required for approximating histogram distributions using ReLU networks; Yang et al. [34] showed that deep generative networks can transform low-dimensional inputs into high-dimensional distributions with minimal Wasserstein distance; and Yarotsky [35] further established that even two-parameter models can learn d-dimensional distributions with arbitrarily high success probability under gradient flow.

These theoretical foundations confirm that neural networks possess the capability to overcome the dimensionality constraints of classical sampling methods. Building upon these insights, this work introduces a GAN-based Bernoulli distribution sampler that bridges the gap between theoretical expressivity and practical cryptographic application, enabling efficient low-to-high-dimensional sampling for secure pseudo-random number generation.

3. Our scheme

In this section, we present the core components of our construction. We begin by introducing the GAN-based Bernoulli Noise Expansion Sampler. Building upon this sampler, we then formulate a novel variant of the LPN problem that incorporates the structural properties of our generative model. Finally, we propose a new PRNG architecture based on these foundational elements.

3.1. Main idea

The PRNG structure depicted in Fig. 1 is based on the hardness of the LPN problem. Specifically, given a secret matrix M, an initial secret vector $\mathbf{v}_0$, and an error vector $\mathbf{e}_0$, the expression $M^T\mathbf{v}_0 + \mathbf{e}_0$ can be used to generate the input for the next iteration. However, merely iterating this process does not suffice to construct a fully functional PRNG, as one of the fundamental requirements of a PRNG is that its output length must exceed that of the input.

When $n \gg m$, the resulting output of length n can be partitioned into three components: m bits are used to derive the next state vector $\mathbf{v}_{i+1}$, a subset of bits is selected as the output bitstream z_i, and the remaining portion r_i is transformed into the subsequent error vector $\mathbf{e}_{i+1}$. Consequently, a critical component in such PRNG constructions is a sampling mechanism capable of converting a uniformly distributed short input into a long output of length n following a Bernoulli distribution with parameter p. This transformation constitutes a central challenge in the design of such LPN-based PRNGs. In this context, we refer to such a sampler as a Bernoulli Noise Expansion Sampler, due to the unique re-

quirement that the output length significantly exceeds the length of the uniformly distributed input.

In 2008, Applebaum et al. [36] pointed out that generating Bernoulli-distributed noise sequences with specific statistical properties from short uniform random seeds is both theoretically interesting and practically relevant. They proposed that such noise expansion samplers can be realized through suitable pseudorandom generator constructions. In 2021, Sonia Bogos et al. [37] introduced a coding-theoretic approach, further extending the efficient implementation of Bernoulli noise expansion samplers in symmetric encryption settings. These approaches primarily rely on cryptographic constructions or algebraic coding techniques, which may impose certain limitations on design flexibility and adaptability.

Recently, neural networks have demonstrated strong capabilities in modeling complex data distributions and learning latent features [32,33,35]. Inspired by these developments, this paper proposes a construction methodology for Bernoulli noise expansion samplers based on deep neural networks. The proposed method leverages adversarial training mechanisms to enable the generator to automatically learn and emulate the statistical properties of the target Bernoulli distribution, thereby producing high-dimensional binary noise sequences.

It is important to emphasize that the samples generated by the proposed method are intended to follow a product Bernoulli distribution with independent coordinates across all dimensions. However, the success probability of each bit is not strictly equal to the fixed noise parameter p assumed in the standard LPN problem; rather, it may vary within a small neighborhood around p. While this characteristic slightly deviates from the traditional i.i.d. LPN setting, we provide a theoretical analysis in subsequent sections showing that such distributions can still induce computationally hard LPN instances under explicit deviation bounds. Therefore, the proposed GAN-based noise expansion sampler can serve as a noise source in the LPN-style PRNG construction considered in this paper.

3.2. Bernoulli noise expansion sampler based on GAN

This section elaborates on the design of the Bernoulli Noise Expansion Sampler, a core component of our PRNG, which is constructed atop the GAN framework. As illustrated in Fig. 2, the sampler comprises two primary neural networks: a Generator and a Discriminator, which engage in an adversarial training process. The objective is to use this framework to learn the statistical structure of a target Bernoulli distribution $\mathrm{Ber}(p)^n$, thereby enabling the generation of high-dimensional binary vectors from a low-dimensional random seed. The use of the Wasserstein-1 distance, stabilized by a gradient penalty, improves training stability compared to conventional GANs and supports more reliable approximation of the target noise distribution.

The Generator Network G, shown on the left side of Fig. 2, is defined as a mapping $G : \{0,1\}^{\ell} \to \{0,1\}^n$. It transforms an ℓ-dimensional uniformly distributed binary seed into an n-dimensional output vector $\mathbf{X} = (X_1, X_2, \ldots, X_n)$. A key architectural feature is the use of independent subnetworks $g_1, g_2, \ldots, g_n$, each responsible for generating a single output bit X_i. This design is intended to promote statistical independence among the output bits, which is important for the subsequent LPN-based construction. The generator is trained so that each X_j approximately follows a Bernoulli distribution with parameter close to p. The Discriminator Network D acts as a critic that guides the generator's training. Instead of binary classification, it outputs a real-valued score that estimates the authenticity of an input sample. By evaluating generated and reference samples, the discriminator provides gradient information that encourages the generator's output distribution to match the target distribution in both marginal behavior and dependence structure.

Notably, the proposed sampler undergoes a one-time offline training phase. It does not require pre-existing historical cryptographic datasets or the direct ingestion of real-time physical randomness during training. The training dataset is composed entirely of synthetic samples, which are dynamically generated during each training step by sampling from the theoretical Bernoulli distribution $\mathrm{Ber}(p)^n$ using the standard pseudorandom number generator provided by PaddlePaddle in our experiments. This approach allows the generator to learn from an idealized reference distribution rather than from potentially biased real-world data sources. Once trained, the generator's parameters are fixed for use in the subsequent construction.

3.2.1. Generator design

This section presents a formal definition and key properties of the generator G, which is used to construct noise samples. The goal is to design a generator that produces binary outputs approximating the target Bernoulli product distribution used in the subsequent LPN-style construction.

To achieve this objective, we define the generator through four aspects:

(1) Formal Definition

For notational simplicity, the sampler is described here in single-call form, where one invocation of G produces one n-bit output vector. In the actual training and implementation pipeline, however, the generator is typically evaluated on a minibatch of independent latent seeds $\{\mathbf{z}^{(b)}\}_{b=1}^{B}$, producing a batch of output vectors $\{\mathbf{x}^{(b)}\}_{b=1}^{B}$ in parallel. This batched evaluation is an implementation detail and does not change the single-call formal definition above.

Let λ be the security parameter. We define the generator in single-call form as a mapping

$$G : \{0,1\}^{\ell} \to \{0,1\}^n.$$

For an input seed $\mathbf{z} \sim U(\{0,1\}^{\ell})$, the output vector is

$$\mathbf{x} = G(\mathbf{z}) = (X_1, X_2, \ldots, X_n) \in \{0,1\}^n.$$

We implement G via n independently parameterized coordinate generators $\{g_i\}_{i=1}^{n}$. In the analysis used later for independence, the aggregate seed $\mathbf{z}$ is understood as being internally parsed into coordinate-level latent components

$$\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_n), \qquad \mathbf{z}_i \in \{0,1\}^{\ell_i}, \qquad \sum_{i=1}^{n} \ell_i = \ell,$$

where the components are mutually independent. The i-th output bit is then written as

$$X_i := G_i(\mathbf{z}_i), \qquad i \in [n].$$

Equivalently,

$$G(\mathbf{z}) = \big(G_1(\mathbf{z}_1), G_2(\mathbf{z}_2), \ldots, G_n(\mathbf{z}_n)\big).$$

Here, $\ell = \mathrm{poly}(\lambda)$ denotes the total length of the aggregate latent seed, and $n = \mathrm{poly}(\lambda)$ is the output dimension. Each $X_i \in \{0,1\}$ is a Bernoulli random variable with success probability $p_i \in [\delta_1, \delta_2]$, where

$$0 < \delta_1 \le \delta_2 < \frac{1}{2}.$$

Optionally, if a target rate p is specified, we measure the deviation by

$$\epsilon_{\mathrm{bias}} := \max_{i \in [n]} |p_i - p|.$$

We further require the generator to satisfy two essential properties:

1) Independence: The output bits of a single generated vector satisfy

$$P(\mathbf{x}) = \prod_{i=1}^{n} P(X_i = x_i)$$

under the conditions stated in Theorem 1. Any non-ideal deviation from product structure is quantified explicitly by ϵ_{ind} in the subsequent analysis.

2) Distribution Fidelity: For every $i \in [n]$, it holds that

$$p_i := \Pr[X_i = 1] \in [\delta_1, \delta_2].$$

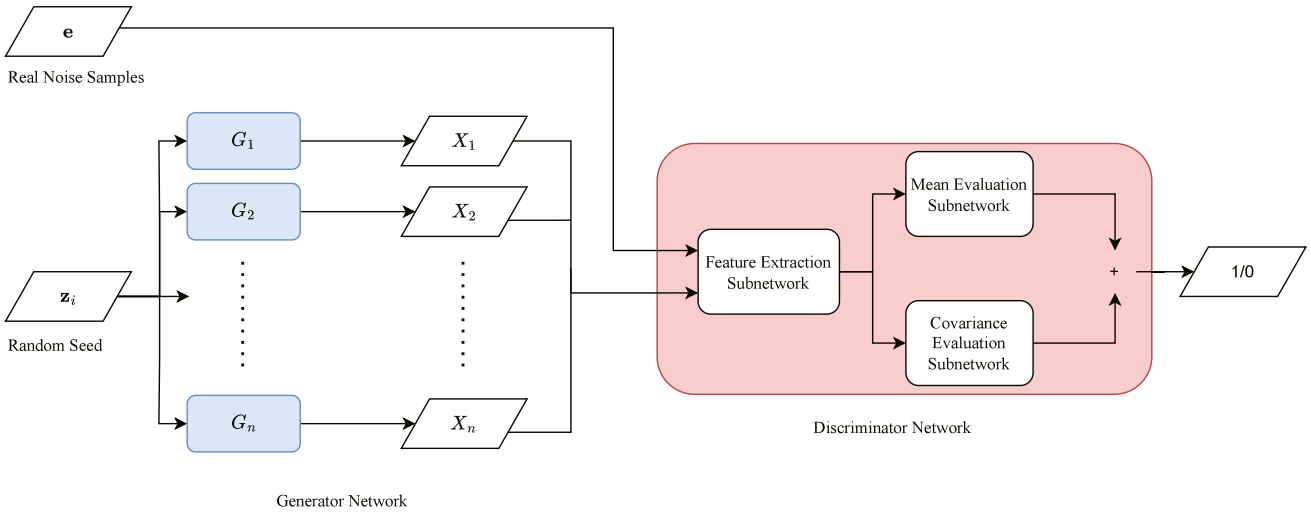

Fig. 2. Bernoulli Noise Expansion Sampler Structure.

Optionally, relative to a target Bernoulli rate p, we track the mismatch through ϵ_{bias} as defined above. These deviations are carried explicitly in the subsequent security bounds.

(2) Independent Subnetwork Architecture

To enhance modeling capacity while avoiding rigid depth constraints, we propose an architecture composed of independently parameterized subnetworks. Each output bit $X_i \in \{0,1\}$ is obtained from a dedicated subgenerator g_i, with $L = L(\lambda)$ hidden layers. This modular structure allows for dynamic depth adjustment based on the security parameter λ, balancing computational efficiency and representational power.

Specifically, the i-th subgenerator is defined as

$$G_i : \{0,1\}^{\ell_i} \to \{0,1\}, \quad \mathbf{z}_i \mapsto G_i(\mathbf{z}_i), \qquad X_i = G_i(\mathbf{z}_i),$$

implemented as a feedforward neural network (with L hidden layers), structured as:

$$\begin{aligned} G_i(\mathbf{z}_i) &= \text{BinarySTE}\big(\mathbf{w}_{L+1}^{(i)} \cdot \text{LN}(H_L^{(i)})\big) \\ H_L^{(i)} &= \sigma_L^{(i)}\big(H_{L-1}^{(i)}\big) \\ &\vdots \\ H_2^{(i)} &= \sigma_2^{(i)}\Big(\mathbf{W}_2^{(i)} \cdot \text{LN}\big(H_1^{(i)}\big)\Big) \\ H_1^{(i)} &= \sigma_1^{(i)}\big(\mathbf{W}_1^{(i)} \cdot \mathbf{z}_i\big) \end{aligned}$$

where $\mathbf{z}_i \in \{0,1\}^{\ell_i}$ is the input latent component for the i-th coordinate generator, $\mathbf{W}_t^{(i)} \in \mathbb{R}^{d_t^{(i)} \times d_{t-1}^{(i)}}$ is the weight matrix at hidden layer t, with $d_0^{(i)} = \ell_i$, and $\mathbf{w}_{L+1}^{(i)} \in \mathbb{R}^{1 \times d_L^{(i)}}$ is the output-layer weight vector. The activation $\sigma_t^{(i)}(\cdot)$ is chosen from $\{\text{Hardtanh}, \text{ReLU}, \text{LeakyReLU}\}$, $\text{LN}(\cdot)$ denotes layer normalization, and $\text{BinarySTE}(\cdot)$ is applied to map the final continuous output to a bit.

Due to the absence of shared parameters across subnetworks — i.e., for any $i \neq j$:

$$\left(\bigcup_{t=1}^{L+1} \mathbf{W}_t^{(i)}\right) \cap \left(\bigcup_{t=1}^{L+1} \mathbf{W}_t^{(j)}\right) = \emptyset,$$

the coordinate generators are implemented independently. This non-sharing architecture is intended to reduce direct coupling between different output coordinates. Any residual deviation from an ideal product Bernoulli distribution is not assumed away, but is instead quantified explicitly in the subsequent security analysis.

In practice, the network depth $L(\lambda)$ can be adaptively set according to the security level. For lower security settings, $L(\lambda) = 2$ or 3 suffices, whereas for higher levels, $L(\lambda) = O(\log \lambda)$ is preferred to enhance model expressiveness and robustness. Additionally, the width $d_t^{(i)}$ of each layer can be adjusted dynamically based on training performance, subject to

$$d_t^{(i)} = \Omega(\log \lambda), \quad \forall t \in [L],$$

ensuring that the generator operates within polynomial time.

(3) Multi-constraint Loss Function

To guide the generator in learning the target distribution, we introduce three types of loss functions and optimize them using an adaptive weighting mechanism. Since the generated noise vector is $\mathbf{x} = G(\mathbf{z}) = (X_1, \ldots, X_n)$, the losses are computed on $\mathbf{x}$, and in practice are estimated empirically over minibatches.

1) Adversarial Loss:

$$\mathcal{L}_{\text{adv}} = -\mathbb{E}_{\mathbf{z} \sim U(\{0,1\}^{\ell})}[D(G(\mathbf{z}))] + \mathbb{E}_{\mathbf{e} \sim \text{Ber}(p)^n}[D(\mathbf{e})],$$

where the critic $D : \{0,1\}^n \to \mathbb{R}$ satisfies the Lipschitz constraint:

$$|D(\mathbf{x}) - D(\mathbf{y})| \leq \|\mathbf{x} - \mathbf{y}\|_1.$$

2) Mean Constraint Loss:

$$\mathcal{L}_{\text{mean}} = \left(\frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[X_i] - p\right)^2.$$

3) Covariance Constraint Loss:

$$\mathcal{L}_{\text{cov}} = \frac{1}{n^2}\sum_{i \neq j} \big(\text{Cov}(X_i, X_j)\big)^2,$$

where

$$\text{Cov}(X_i, X_j) = \mathbb{E}[X_i X_j] - \mathbb{E}[X_i]\mathbb{E}[X_j].$$

This term penalizes pairwise correlations among dimensions and is intended to promote approximate independence.

(4) Dynamic Weight Update Rule

To balance the influence of different loss components, we employ the following adaptive weighting mechanism:

$$w_{\text{mean}}^{(t+1)} = \frac{\alpha}{\mathcal{L}_{\text{mean}}^{(t)} + \beta}, \quad w_{\text{cov}}^{(t+1)} = \frac{\gamma}{\mathcal{L}_{\text{cov}}^{(t)} + \delta},$$

where $\alpha, \beta, \gamma, \delta$ are hyperparameters and t denotes the training epoch.

This mechanism ensures that when a certain loss decreases, its corresponding weight increases automatically, preventing premature convergence to local optima.

3.2.2. *Discriminator design*

The discriminator D is defined as a mapping function:

$$D : \{0,1\}^n \to \mathbb{R}, \quad \mathbf{x} \mapsto r = D(\mathbf{x}),$$

where the input $\mathbf{x} \in \{0,1\}^n$ may originate from either the real distribution or the generator output $\mathbf{x} = G(\mathbf{z})$ with $\mathbf{z} \sim U(\{0,1\}^{\ell})$; the output $r \in \mathbb{R}$ is a real-valued score that reflects the "authenticity" of the input sample.

In the batched training implementation, the discriminator is evaluated on a minibatch of generated and reference samples. For clarity, however, the formal definition and losses below are written in single-sample form.

(1) Network Architecture

The discriminator adopts a dual-head structure, composed of three functional subnetworks responsible for feature extraction, mean evaluation, and covariance evaluation, respectively. The overall architecture can be formally defined as follows:

1) Feature Extraction Subnetwork

Define the mapping $f_{\text{feat}} : \{0,1\}^n \to \mathbb{R}^d$, implemented as a fully connected neural network (FCNN):

$$f_{\text{feat}}(\mathbf{x}) = \mathbf{W}_{L_D} \cdot \sigma_{L_D-1}\big(\cdots \sigma_1\big(\mathbf{W}_1 \cdot \mathbf{x}\big)\big),$$

where $\mathbf{W}_t \in \mathbb{R}^{d_t \times d_{t-1}}$ is the weight matrix at layer t, with $d_0 = n$, $d_{L_D} = d$; $\sigma_t(\cdot)$ denotes the LeakyReLU activation function:

$$\sigma_t(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}, \quad \alpha \in (0,1).$$

This subnetwork performs nonlinear transformations to map the input into a latent space and extract higher-order statistical features.

2) Mean Evaluation Subnetwork

Define the mapping $f_{\text{mean}} : \mathbb{R}^d \to \mathbb{R}$, structured as:

$$f_{\text{mean}}(\mathbf{h}) = \text{Tanh}\big(\mathbf{W}_m \cdot \mathbf{h} + b_m\big),$$

where $\mathbf{W}_m \in \mathbb{R}^{1 \times d}$ and $b_m \in \mathbb{R}$ are learnable parameters; $\text{Tanh}(\cdot)$ is the hyperbolic tangent activation function, yielding outputs in the range $[-1, 1]$.

This subnetwork evaluates whether the feature representation is consistent with the target Bernoulli mean behavior.

3) Covariance Evaluation Subnetwork

Define the mapping $f_{\text{cov}} : \mathbb{R}^d \to \mathbb{R}$, structured as:

$$f_{\text{cov}}(\mathbf{h}) = \text{Tanh}\big(\mathbf{W}_c \cdot \mathbf{h} + b_c\big),$$

where $\mathbf{W}_c \in \mathbb{R}^{1 \times d}$ and $b_c \in \mathbb{R}$ are learnable parameters.

This subnetwork evaluates whether the feature representation is consistent with weak inter-dimensional dependence.

(2) Output Mechanism and Loss Function

The final output of the discriminator is the algebraic sum of the mean and covariance evaluations:

$$D(\mathbf{x}) = f_{\text{mean}}(f_{\text{feat}}(\mathbf{x})) + f_{\text{cov}}(f_{\text{feat}}(\mathbf{x})).$$

This dual-head design enhances the discriminator's ability to evaluate input samples along both mean alignment and covariance minimization dimensions.

The discriminator is trained using a WGAN-GP-style loss function:

$$\mathcal{L}_D = \mathbb{E}_{\mathbf{e}\sim \text{Ber}(p)^n}[D(\mathbf{e})] - \mathbb{E}_{\mathbf{z}\sim U(\{0,1\}^{\ell})}[D(G(\mathbf{z}))] - \lambda \cdot \mathbb{E}_{\hat{\mathbf{x}}\sim \mathcal{P}_{\text{mix}}}\Big[\big(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1\big)^2\Big],$$

where $\lambda > 0$ is a hyperparameter controlling the strength of the gradient constraint; $\mathcal{P}_{\text{mix}}$ denotes the linear interpolation between real samples $\mathbf{e}$ and generated samples $G(\mathbf{z})$, defined as

$$\hat{\mathbf{x}} = \epsilon \cdot \mathbf{e} + (1-\epsilon) \cdot G(\mathbf{z}), \quad \epsilon \sim \text{Uniform}([0,1]).$$

3.2.3. Training design

This study constructs the Bernoulli noise expansion sampler within a WGAN-GP-based framework. The model consists of a generator G and a discriminator D, which are trained collaboratively through an adversarial learning mechanism. The objective is to enable the generator to approximate the target Bernoulli product distribution as closely as possible.

Formally, the generator is defined in single-call form as a mapping. In the actual training implementation, however, we use batched evaluation: each update step operates on a minibatch of independent latent seeds $\{\mathbf{z}^{(b)}\}_{b=1}^{B}$ and produces a corresponding minibatch of output vectors $\{\mathbf{x}^{(b)}\}_{b=1}^{B}$. This vectorized computation is an implementation detail and does not alter the single-call formal definition adopted in the theoretical analysis.

As shown in Algorithm 1, the overall training process follows the standard alternating optimization paradigm of GANs, in which the parameters of the discriminator and generator are updated in turn. Specifically, for each generator update, the discriminator is typically updated multiple times. This iterative procedure improves the discriminative capability of D, allowing it to provide more informative gradients to guide the training of G.

Algorithm 1 WGAN-GP-based Bernoulli Noise Expansion Sampler Training Algorithm.

Require: Initial generator parameters θ^0, initial discriminator parameters ϕ^0, number of training epochs T, number of discriminator updates per generator step n_D, learning rates η_D, η_G, gradient penalty coefficient λ_{GP}, minibatch size B, initial loss weights $w_{\text{mean}}^{(0)}, w_{\text{cov}}^{(0)}$

Ensure: Trained generator parameters θ^*, discriminator parameters ϕ^*

Initialize $\theta \leftarrow \theta^0$, $\phi \leftarrow \phi^0$
for $t = 0$ **to** $T-1$ **do**
  // **Discriminator updates**
  for $j = 0$ **to** $n_D - 1$ **do**
    Sample latent seeds $\{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(B)}\}$ independently from $U(\{0,1\}^{\ell})$
    Generate fake samples $\mathbf{X}_{\text{fake}} = \{G_\theta(\mathbf{z}^{(i)})\}_{i=1}^{B}$
    Sample real reference samples $\mathbf{X}_{\text{real}} = \{\mathbf{e}^{(i)}\}_{i=1}^{B}$ independently from $\text{Ber}(p)^n$
    Construct interpolated samples for gradient penalty
    Compute the discriminator loss $\mathcal{L}_D$ as defined above
    Compute gradient $\nabla_\phi \mathcal{L}_D$
    Update discriminator parameters:

$$\phi \leftarrow \phi - \eta_D \nabla_\phi \mathcal{L}_D$$

  end for
  // **Generator update**
  Sample latent seeds $\{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(B)}\}$ independently from $U(\{0,1\}^{\ell})$
  Generate fake samples $\mathbf{X}_{\text{fake}} = \{\mathbf{x}^{(i)}\}_{i=1}^{B}$, where $\mathbf{x}^{(i)} = G_\theta(\mathbf{z}^{(i)})$
  Compute the adversarial loss $\mathcal{L}_{\text{adv}}$ as defined above
  Compute the mean constraint loss $\mathcal{L}_{\text{mean}}$ as defined above
  Compute the covariance constraint loss $\mathcal{L}_{\text{cov}}$ as defined above
  Update the adaptive weights $w_{\text{mean}}^{(t+1)}$ and $w_{\text{cov}}^{(t+1)}$ as defined above
  Compute the total generator loss $\mathcal{L}_G$
  Compute gradient $\nabla_\theta \mathcal{L}_G$
  Update generator parameters:

$$\theta \leftarrow \theta - \eta_G \nabla_\theta \mathcal{L}_G$$

end for
$\theta^* \leftarrow \theta$, $\phi^* \leftarrow \phi$
return θ^*, ϕ^*

3.2.4. Theoretical guarantee of essential properties

This section provides a rigorous theoretical foundation for the two essential properties of our proposed GAN-based Bernoulli noise sampler: Statistical Independence and Distribution Fidelity. The two properties below are stated with respect to this output vector $\mathbf{x}$, which is the object used by the subsequent cryptographic construction.

(1) Guarantee of Statistical Independence

The independence of the output bits is promoted by the sampling mechanism and architectural design, rather than being assumed solely from training behavior. In the analysis below, the single aggregate latent

seed $\mathbf{z}$ used in one call to G is viewed as being internally decomposed into coordinate-level latent components $(\mathbf{z}_1, \dots, \mathbf{z}_n)$, and this property is obtained from the mutual independence of these components together with deterministic inference.

Theorem 1 (Independence). *Suppose that in the sampling analysis: (a) the aggregate latent seed $\mathbf{z}$ can be internally parsed as $(\mathbf{z}_1, \dots, \mathbf{z}_n)$, where the coordinate-level latent components are mutually independent and uniformly distributed; and (b) each coordinate generator is deterministic at inference time. Then the output bits $X_1, X_2, \dots, X_n$ are mutually independent, i.e.,*

$$P(\mathbf{x}) = \prod_{i=1}^{n} P(X_i = x_i).$$

Proof. Write

$$\mathbf{x} = G(\mathbf{z}) = (X_1, \dots, X_n),$$

where each coordinate is given by

$$X_i = G_i(\mathbf{z}_i).$$

By assumption (b), each coordinate output is a deterministic function of its coordinate-level latent component. By assumption (a), the latent components $\mathbf{z}_1, \dots, \mathbf{z}_n$ are mutually independent. A deterministic function applied coordinate-wise to independent inputs yields independent outputs. Hence $X_1, \dots, X_n$ are mutually independent and the joint distribution factorizes.

If an implementation deviates from ideal independent latent components or introduces shared randomness at inference time, we quantify the resulting deviation from product structure by

$$\epsilon_{\text{ind}} := \Delta\left(P_{\mathbf{x}},\ \prod_{i=1}^{n} P_{X_i}\right),$$

where $\Delta(\cdot,\cdot)$ denotes statistical distance (total variation).

(2) Guarantee of Distribution Fidelity

In our setting, distribution fidelity is naturally expressed by allowing non-identical Bernoulli rates across coordinates while keeping each coordinate close to a desired operating regime.

First, define the marginal noise rate of each coordinate by

$$p_i := \Pr[X_i = 1] = \mathbb{E}[X_i].$$

We require these rates to be bounded away from 0 and from $1/2$, i.e., there exist parameters $0 < \delta_1 \le \delta_2 < 1/2$ such that

$$p_i \in [\delta_1, \delta_2], \qquad \forall i \in [n].$$

Optionally, to target a global mean noise rate p, we measure the deviation

$$\epsilon_{\text{bias}} := \max_{i\in[n]} |p_i - p|,$$

which is estimated from samples in our experiments. Importantly, the subsequent security reduction does not require ϵ_{bias} to be negligible; it is carried explicitly as an additive term. □

Theorem 2 (Distribution Fidelity: realizability of non-identical Bernoulli rates). *Fix any target rate vector $(p_1, \dots, p_n)$ with $p_i \in (0,1)$. There exists a choice of parameters for the coordinate generators $\{G_i\}_{i=1}^n$ such that, under an internal parsing*

$$\mathbf{z} = (\mathbf{z}_1, \dots, \mathbf{z}_n), \qquad \mathbf{z}_i \in \{0,1\}^{\ell_i},$$

for every $i \in [n]$,

$$\left|\mathbb{E}[X_i] - p_i\right| \le \epsilon_G^{(i)}, \qquad \epsilon_G^{(i)} \le 2^{-\ell_i}.$$

Defining

$$\epsilon_G := \max_{i\in[n]} \epsilon_G^{(i)},$$

we obtain

$$\left|\mathbb{E}[X_i] - p_i\right| \le \epsilon_G, \qquad \forall i \in [n].$$

In particular, if $p_i \in [\delta_1, \delta_2]$ for all i, then

$$\mathbb{E}[X_i] \in [\delta_1 - \epsilon_G, \delta_2 + \epsilon_G], \qquad \forall i \in [n].$$

The proof of Theorem 2 is based on the following explicit Bernoulli sampler from a uniform coordinate-level latent component.

Lemma 1 (Finite-precision Bernoulli sampling). *Let Z be uniform over $\{0,1\}^{\ell'}$, interpreted as an integer in $\{0,1,\dots,2^{\ell'}-1\}$. Fix $p \in (0,1)$ and define the threshold $T := \lfloor p \cdot 2^{\ell'} \rfloor$. Let*

$$X := \mathbb{I}[Z < T] \in \{0,1\}.$$

Then $X \sim Ber(p')$ with $p' = T/2^{\ell'}$ and

$$|p' - p| \le 2^{-\ell'}.$$

Equivalently, $|\mathbb{E}[X] - p| \le 2^{-\ell'}$.

Lemma 2 (Proof of Lemma 1). *Since Z is uniform on $\{0,\dots,2^{\ell'}-1\}$, we have*

$$\Pr[X = 1] = \Pr[Z < T] = \frac{T}{2^{\ell'}} = p'.$$

Moreover, by definition of $T = \lfloor p \cdot 2^{\ell'} \rfloor$, it holds that

$$0 \le p - \frac{T}{2^{\ell'}} < 2^{-\ell'},$$

so $|p' - p| \le 2^{-\ell'}$. □

Proof of Theorem 2. For each $i \in [n]$, implement the coordinate generator g_i so that its output

$$X_i = G_i(\mathbf{z}_i)$$

realizes the threshold sampler in Lemma 1 with parameter $p = p_i$ and coordinate-level latent component length $\ell' = \ell_i$. The threshold comparison can be realized by a polynomial-size feedforward network (equivalently, by a polynomial-size Boolean circuit embedded into our BinarySTE-based architecture), yielding a valid parameter setting for g_i within our model class. By Lemma 1, the sampling rule implies

$$|\mathbb{E}[X_i] - p_i| \le 2^{-\ell_i}.$$

Define

$$\epsilon_G := \max_{i\in[n]} 2^{-\ell_i}$$

as the finite-precision implementation loss. Then, for every $i \in [n]$,

$$|\mathbb{E}[X_i] - p_i| \le \epsilon_G.$$

In the subsequent security bounds, ϵ_G is treated as an explicit additive degradation term. □

This existence statement is consistent with standard expressivity results for deep networks (e.g., Yarotsky [38]): step-like/comparator functions (and, more generally, piecewise-constant mappings on bounded domains) can be represented or approximated by networks of polynomial size with depth logarithmic in the approximation error. In practice we do not hard-code the comparator; instead, WGAN-GP training (together with the mean/covariance constraints) is used to approximate such samplers and reduce the empirically measured deviations in p_i and any residual correlations. The later security reduction remains valid under realistic deviations by carrying those deviations as explicit additive terms. □

Corollary 1. (Closeness to ideal i.i.d. LPN noise and explicit degradation). *Let $p_i = \Pr[X_i = 1]$ and fix a target $p \in (0, 1/2)$. Then*

$$\Delta\big(P_{\mathbf{x}},\ Ber(p)^n\big) \le \epsilon_{ind} + \sum_{i=1}^{n} |p_i - p|.$$

In particular, if $\epsilon_{bias} := \max_i |p_i - p|$, then

$$\Delta\big(P_{\mathbf{x}},\ Ber(p)^n\big) \le \epsilon_{ind} + n \cdot \epsilon_{bias}.$$

Proof. By definition of ϵ_{ind} and the triangle inequality,

$$\Delta\big(P_{\mathbf{x}},\ \mathrm{Ber}(p)^n\big) \le \Delta\left(P_{\mathbf{x}},\ \prod_{i=1}^{n}\mathrm{Ber}(p_i)\right) + \Delta\left(\prod_{i=1}^{n}\mathrm{Ber}(p_i),\ \mathrm{Ber}(p)^n\right)$$
$$= \epsilon_{\text{ind}} + \Delta\left(\prod_{i=1}^{n}\mathrm{Ber}(p_i),\ \prod_{i=1}^{n}\mathrm{Ber}(p)\right).$$

For each coordinate, $\Delta(\mathrm{Ber}(p_i), \mathrm{Ber}(p)) = |p_i - p|$. Using the standard product bound for total variation (e.g., via a coupling/union bound argument),

$$\Delta\left(\prod_{i=1}^{n}\mathrm{Ber}(p_i),\ \prod_{i=1}^{n}\mathrm{Ber}(p)\right) \le \sum_{i=1}^{n}\Delta(\mathrm{Ber}(p_i), \mathrm{Ber}(p)) = \sum_{i=1}^{n}|p_i - p|.$$

This proves the claim. □

Corollary 2. (Baseline hardness via minimum noise rate). *Suppose further that the induced Bernoulli rates satisfy $p_i \in [\delta_1, \delta_2]$ for all $i \in [n]$, where $0 < \delta_1 \le \delta_2 < 1/2$. Consider the (decision) LPN distinguishing problem in which samples are either*

$$(A,\ A\mathbf{s} \oplus \mathbf{e}) \quad \textit{with} \quad e_i \sim \mathrm{Ber}(p_i)\ \textit{independently},$$

or

$$(A,\ \mathbf{u}) \quad \textit{with} \quad \mathbf{u} \sim U(\{0,1\}^n),$$

where A is uniform, $\mathbf{s}$ is uniform, and $\mathbf{e}, \mathbf{u}$ are independent of $(A, \mathbf{s})$. If there exists a PPT distinguisher $\mathcal{A}$ that achieves advantage $\mathrm{Adv}_{\mathcal{A}}$ against the above non-identical-rate LPN distribution, then there exists a PPT distinguisher $\mathcal{B}$ that achieves the same advantage against standard LPN with i.i.d. noise rate δ_1. Consequently, the non-identical-rate setting with all $p_i \ge \delta_1$ is no easier than standard LPN(δ_1).

Proof. Let LPN_{δ_1} denote the standard decision-LPN distribution with i.i.d. noise $e_i \sim \mathrm{Ber}(\delta_1)$, and let UNI denote the uniform distribution $(A, \mathbf{u})$. Algorithm $\mathcal{B}$ is given a challenge sample $(A, \mathbf{b})$ which is drawn either from LPN_{δ_1} or from UNI, and must distinguish which is the case.

$\mathcal{B}$ proceeds as follows. For each $i \in [n]$, define

$$q_i := \frac{p_i - \delta_1}{1 - 2\delta_1}.$$

Since $p_i \in [\delta_1, \delta_2]$ and $\delta_2 < 1/2$, we have $q_i \in [0, 1/2)$. Sample $\mathbf{r} \in \{0,1\}^n$ with independent coordinates $r_i \sim \mathrm{Ber}(q_i)$, and output to $\mathcal{A}$ the pair

$$(A,\ \mathbf{b}'), \qquad \mathbf{b}' := \mathbf{b} \oplus \mathbf{r}.$$

Finally, $\mathcal{B}$ outputs whatever bit $\mathcal{A}$ outputs on input $(A, \mathbf{b}')$.

We now analyze the two cases.

Case 1: $(A, \mathbf{b}) \leftarrow \mathsf{UNI}$. Then $\mathbf{b}$ is uniform over $\{0,1\}^n$. For any fixed $\mathbf{r}$, the map $\mathbf{b} \mapsto \mathbf{b} \oplus \mathbf{r}$ is a permutation of $\{0,1\}^n$, hence $\mathbf{b}'$ is uniform as well. Therefore $(A, \mathbf{b}')$ has exactly the UNI distribution.

Case 2: $(A, \mathbf{b}) \leftarrow \mathsf{LPN}_{\delta_1}$. Then $\mathbf{b} = A\mathbf{s} \oplus \mathbf{e}$ where $e_i \sim \mathrm{Ber}(\delta_1)$ independently. Define $\mathbf{e}' := \mathbf{e} \oplus \mathbf{r}$. Since $\mathbf{e}$ and $\mathbf{r}$ are independent and each has independent coordinates, $\mathbf{e}'$ has independent coordinates as well. Moreover, for each i,

$$\begin{aligned}\Pr[e'_i = 1] &= \Pr[e_i \oplus r_i = 1] \\ &= \delta_1(1 - q_i) + (1 - \delta_1)q_i \\ &= \delta_1 + q_i - 2\delta_1 q_i \\ &= p_i,\end{aligned}$$

where the last equality holds by the definition of q_i. Hence $e'_i \sim \mathrm{Ber}(p_i)$ independently, and therefore

$$\mathbf{b}' = \mathbf{b} \oplus \mathbf{r} = A\mathbf{s} \oplus \mathbf{e} \oplus \mathbf{r} = A\mathbf{s} \oplus \mathbf{e}'$$

is distributed exactly as an LPN sample with independent non-identical noise rates $(p_i)_{i=1}^n$.

Therefore, $\mathcal{B}$ perfectly simulates the input distribution expected by $\mathcal{A}$ in both cases, and we obtain

$$\mathrm{Adv}_{\mathcal{B}}^{\mathsf{LPN}_{\delta_1}} = \mathrm{Adv}_{\mathcal{A}}^{(p_i)}.$$

This proves that distinguishing (or solving) the non-identical-rate LPN instance with all $p_i \ge \delta_1$ is at least as hard as standard LPN(δ_1). □

3.3. Variant of the LPN problem based on bernoulli noise expansion sampler

To adapt the GAN-based Bernoulli noise expansion sampler for cryptographic constructions, we formally integrate it into the LPN framework, defining a new variant—the GAN-LPN problem. The key distinction from the classical LPN problem lies in the source of the noise vector: the latter employs static, independent and identically distributed (i.i.d.) Bernoulli noise $\mathrm{Ber}(p)^n$, whereas the former utilizes a neural generator. Consequently, the noise distribution in GAN-LPN is defined by the parameters and sampling mechanism of this generator.

We define an LPN-like distribution induced by the GAN-based Bernoulli noise expansion sampler, referred to as the GAN-LPN distribution, denoted $D_{\text{GAN-LPN}}$. This distribution consists of all pairs $(A, \mathbf{b})$, where: $A \in \mathbb{F}_2^{n\times m}$ is sampled uniformly at random from $\mathbb{F}_2^{n\times m}$; $\mathbf{sk} \in \mathbb{F}_2^m$ is a secret vector drawn uniformly from $\mathbb{F}_2^m$; and $\mathbf{b} \in \mathbb{F}_2^n$ is computed via the linear transformation

$$\mathbf{b} = A \cdot \mathbf{sk} \oplus \mathbf{e},$$

where $\oplus$ denotes addition over $\mathbb{F}_2$.

In this definition, the noise vector $\mathbf{e} \in \{0,1\}^n$ is not generated from a traditional i.i.d. Bernoulli source, but is produced by the generator introduced in Section 3.2.1. Concretely, let

$$G : \{0,1\}^{\ell} \to \{0,1\}^n$$

be the generator, and let $\mathbf{z} \xleftarrow{\$} \{0,1\}^{\ell}$ be a uniformly sampled latent seed. We define

$$\mathbf{e} := G(\mathbf{z}) = (X_1, \ldots, X_n).$$

For simplicity, we describe the sampler here in single-call form. In implementation, multiple independent latent seeds may be processed in parallel in batched form, but each call to G still corresponds to one generated noise vector in $\{0,1\}^n$. The single seed $\mathbf{z}$ should therefore be interpreted as the aggregate latent randomness associated with one such generated vector.

We impose the following two constraints on the induced noise bits:

(1) **Statistical Independence.** Under the conditions stated in Theorem 1, the corresponding noise bits $X_1, \ldots, X_n$ are mutually independent, i.e.,

$$P(\mathbf{e}) = \prod_{i=1}^{n} P(X_i = e_i).$$

For non-ideal implementations, any deviation from product structure is quantified explicitly by

$$\epsilon_{\text{ind}} := \Delta\left(P_{\mathbf{e}},\ \prod_{i=1}^{n} P_{X_i}\right),$$

where $\Delta(\cdot,\cdot)$ denotes statistical distance.

(2) **Distributional Fidelity (bounded, non-identical Bernoulli rates).** Each bit X_i follows a Bernoulli distribution with some (possibly non-identical) rate

$$p_i := \Pr[X_i = 1],$$

and we require these rates to lie in a bounded interval

$$p_i \in [\delta_1, \delta_2], \qquad 0 < \delta_1 \le \delta_2 < 1/2.$$

Optionally, if a global target rate p is specified, we measure the deviation by

$$\epsilon_{\text{bias}} := \max_{i\in[n]} |p_i - p|,$$

which is estimated empirically; any such deviation is carried as an explicit additive degradation term in the later security reduction.

The core innovation of this construction is the replacement of the static Bernoulli noise source in the classical LPN problem with a trainable noise expansion sampler. This enables efficient generation of high-dimensional cryptographic noise from short uniform seeds while preserving the LPN algebraic structure. Moreover, under the above constraints, the resulting GAN-LPN instantiation retains a clear baseline

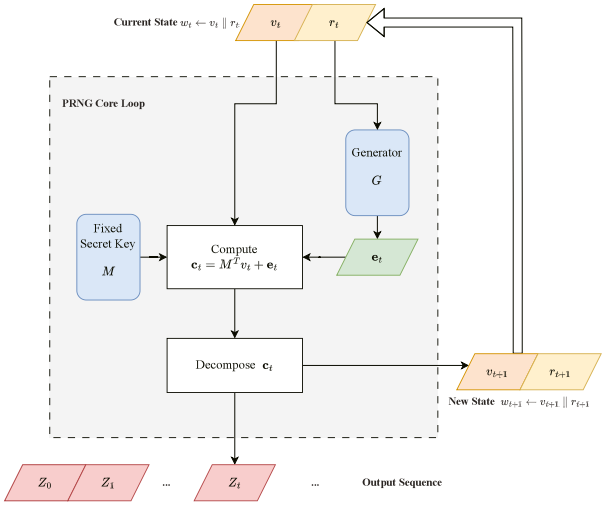

Fig. 3. Overall Architecture of Proposed GAN-LPN PRNG.

hardness guarantee: since all noise rates satisfy $p_i \geq \delta_1$ and the bits are independent (or approximately independent up to the explicitly tracked term ϵ_{ind}), the induced problem is no easier than standard LPN with i.i.d. noise rate δ_1. At the same time, closeness to an i.i.d. $\text{Ber}(p)^n$ noise source can be quantified via ϵ_{bias} together with ϵ_{ind}, yielding an explicit and verifiable security degradation bound.

3.4. Construction of pseudorandom number generator

Although practical implementations may evaluate BNES on a batch of latent seeds for efficiency, the construction of PRNG is written in single-call form, since each PRNG iteration requires only one generated noise vector $\mathbf{e}_t \in \{0,1\}^n$. In this context, the BNES input seed is understood as the aggregate latent seed used for one invocation of the sampler.

As shown in Fig. 3, building upon the GAN-LPN noise distribution defined in the previous subsection, we propose a stateful PRNG that iteratively updates an internal state while injecting Bernoulli noise generated by our Bernoulli Noise Expansion Sampler. At each iteration, the construction computes an LPN-style noisy linear map and then deterministically parses the result into the next state and an output block.

As shown in Fig. 4, its key modules are as follows:

Parameters and state. Let $n \gg m$ and let ℓ denote the BNES seed length. Define the per-round output length

$$\mu := n - m - \ell \qquad (\mu \geq 1).$$

The secret key is a fixed binary matrix $M \in \mathbb{F}_2^{m \times n}$. The evolving internal state is $w_t \in \{0,1\}^{m+\ell}$, parsed as

$$w_t = v_t \| r_t,$$

where $v_t \in \{0,1\}^m$ is the internal secret state and $r_t \in \{0,1\}^\ell$ is the BNES input seed for round t.

All additions are over $\mathbb{F}_2$ (i.e., XOR). We use 1-based indexing for bit vectors: for $\mathbf{c} \in \{0,1\}^n$, the notation $\mathbf{c}[a.b]$ denotes the contiguous substring $(\mathbf{c}[a], \ldots, \mathbf{c}[b])$.

BNES interface and measurable noise parameters. At each round, BNES outputs a noise vector

$$\mathbf{e}_t \leftarrow \text{BNES}(r_t) \in \{0,1\}^n.$$

Let $\mathbf{e}_t = (X_{t,1}, \ldots, X_{t,n})$. The induced noise distribution is characterized by (i) a (possibly non-identical) per-coordinate Bernoulli rate

$$p_i := \Pr[X_{t,i} = 1] \in [\delta_1, \delta_2], \qquad 0 < \delta_1 \leq \delta_2 < 1/2,$$

and (ii) an explicit deviation from product structure

$$\epsilon_{\text{ind}} := \Delta\left(P_{\mathbf{e}_t}, \prod_{i=1}^{n} P_{X_{t,i}}\right).$$

Optionally, for a global target rate $p \in (0, 1/2)$ we measure $\epsilon_{\text{bias}} := \max_i |p_i - p|$ and carry it explicitly as a degradation term in the security analysis; we do not require ϵ_{bias} to be negligible.

Algorithm 2 GAN-LPN PRNG Construction.

Require: Secret key matrix $M \in \mathbb{F}_2^{m \times n}$, initial seed $seed \in \{0,1\}^{m+\ell}$
Ensure: Pseudorandom sequence $\{z_t\}_{t=0}^{\infty}$, where each $z_t \in \{0,1\}^\mu$ and $\mu = n - m - \ell$
1: **Parameter Setup**: choose $n \gg m$, choose ℓ such that $\mu = n - m - \ell \geq 1$
2: **State Initialization**: set $v_0 \leftarrow seed[1.m]$, $r_0 \leftarrow seed[m+1.m+\ell]$
3: $t \leftarrow 0$
4: **while** Need to generate pseudorandom numbers **do**
5: **Noise generation**: $\mathbf{e}_t \leftarrow \text{BNES}(r_t) \in \{0,1\}^n$
6: **Core computation**: $\mathbf{c}_t \leftarrow M^\top v_t \oplus \mathbf{e}_t \in \{0,1\}^n$
7: **Decompose (unambiguous parsing)**: $v_{t+1} \leftarrow \mathbf{c}_t[1.m]$; $r_{t+1} \leftarrow \mathbf{c}_t[m+1.m+\ell]$; $z_t \leftarrow \mathbf{c}_t[m+\ell+1.n]$
8: **State update**: $w_{t+1} \leftarrow v_{t+1} \| r_{t+1}$
9: **Output**: z_t
10: $t \leftarrow t+1$
11: **end while**

As shown in Algorithm 2, each iteration computes the noisy linear image $\mathbf{c}_t = M^\top v_t \oplus \mathbf{e}_t$ and then deterministically slices $\mathbf{c}_t$ into three contiguous blocks: (i) the next secret state v_{t+1}, (ii) the next BNES seed r_{t+1}, and (iii) the output block z_t of length $\mu = n - m - \ell$. The explicit slicing in line 7 eliminates ambiguity in the parsing rule.

This construction integrates a learned Bernoulli noise sampler with an LPN-style update rule. In the security analysis, the bounded-rate condition $p_i \geq \delta_1$ yields a baseline reduction to standard LPN with noise rate δ_1, while any non-idealities of the learned sampler appear as explicit additive degradation terms via ϵ_{ind} (and optionally ϵ_{bias}).

4. Security analysis

This section presents a formal security analysis of the proposed GAN-LPN PRNG. We first establish a formal framework and security model, then define the computational hardness of the GAN-LPN problem and provide a reduction proof from this problem to the standard LPN problem. Finally, based on the hardness of the GAN-LPN problem, we derive conditional forward-security and seed-irrecoverability guarantees from Theorem 4 together with the additional assumptions below.

4.1. Formal framework and assumptions

4.1.1. Adversary model

We consider an efficient adversary $\mathcal{A}$ that may be either a classical probabilistic polynomial-time (PPT) algorithm or, under the corresponding hardness assumption, a quantum polynomial-time (QPT) algorithm. The adversary can adaptively query the PRNG output and may obtain partial internal states of the system (to analyze forward security) under the partial-state exposure model defined above. Our security objective is to show that, even in the presence of such an adversary, the output of the PRNG is computationally indistinguishable from a truly random sequence. The post-quantum (PQ) interpretation follows the standard reductionist paradigm: Theorem 3 provides a black-box reduction from distinguishing the induced GAN-LPN distribution to solving the underlying LPN problem, and this interpretation extends to QPT adversaries provided that the selected LPN parameters remain hard for QPT algorithms and that no quantum-specific obstruction arises in the reduction. Consequently, under these assumptions, our PRNG inherits the assumed post-quantum hardness baseline of the underlying LPN instance.

4.1.2. Security goals

Pseudorandomness: For any PPT adversary, the advantage in distinguishing the output sequence of the PRNG from a truly random se-

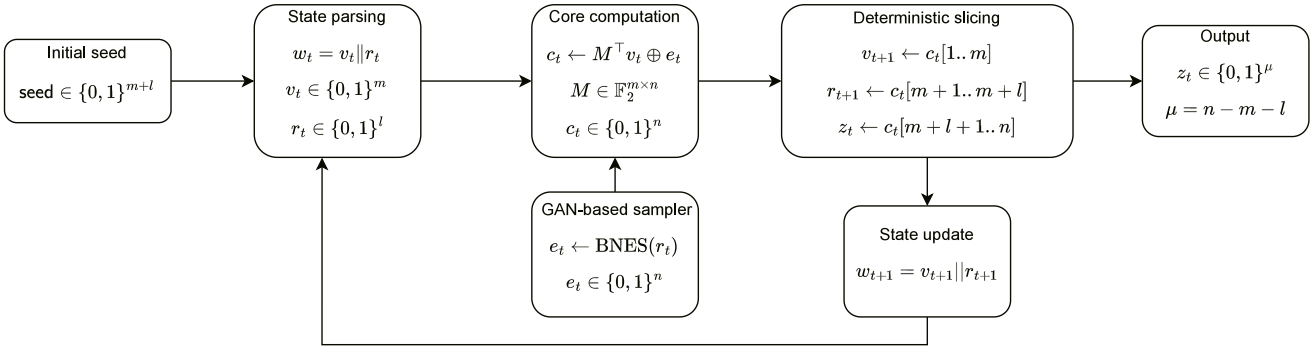

Fig. 4. Block diagram of the proposed GAN LPN PRNG.

quence of the same length is negligible (up to explicit degradation terms induced by non-ideal noise statistics, as quantified below).

Forward Secrecy: If the adversary obtains the internal state w_t at time t, it remains unable to infer any output $z_{t'}$ produced at a prior time $t' < t$.

Seed Irrecoverability: The adversary cannot recover the initial seed from any finite-length output sequence $\{z_i\}$.

4.1.3. Computational assumptions

The foundation of our security rests upon the following computational assumptions, which are stated with respect to efficient adversaries, including those with quantum computational power.

Assumption 1. (Post-quantum hardness of the Standard LPN Problem). For parameters (m, n, δ_1) with $0 < \delta_1 < 1/2$, the standard decisional LPN problem is computationally hard for any quantum polynomial-time adversary $\mathcal{A}_{\mathrm{LPN}}$. Specifically, its distinguishing advantage

$$\mathrm{Adv}_{\mathcal{A}_{\mathrm{LPN}}}^{\mathrm{LPN}(\delta_1)}(\lambda) := \left|\Pr\left[\mathcal{A}_{\mathrm{LPN}}(A, A\mathbf{s} \oplus \mathbf{e}) = 1\right] - \Pr\left[\mathcal{A}_{\mathrm{LPN}}(A, \mathbf{u}) = 1\right]\right| \le \mathrm{negl}(\lambda),$$

where $A \xleftarrow{\$} \mathbb{F}_2^{n\times m}$, $\mathbf{s} \xleftarrow{\$} \mathbb{F}_2^m$, $\mathbf{e} \xleftarrow{\$} \mathrm{Ber}(\delta_1)^n$ (i.i.d.), and $\mathbf{u} \xleftarrow{\$} \mathbb{F}_2^n$.

Assumption 2. (Effectiveness of the Neural Noise Sampler; measurable non-idealities). Let $\mathbf{e} = G(\mathbf{z}) = (X_1, \ldots, X_n)$ denote the n-bit noise vector produced by the trained generator under its prescribed sampling mechanism (Section 3.2.1). Define the per-coordinate rates

$$p_i := \Pr[X_i = 1].$$

We assume there exist $0 < \delta_1 \le \delta_2 < 1/2$ such that for all $i \in [n]$,

$$p_i \in [\delta_1, \delta_2].$$

We further quantify any deviation from full independence by the statistical distance

$$\epsilon_{\mathrm{ind}} := \Delta\left(P_{\mathbf{e}}, \prod_{i=1}^{n} \mathrm{Ber}(p_i)\right).$$

Optionally, for a target rate $p \in (0, 1/2)$ we measure

$$\epsilon_{\mathrm{bias}} := \max_{i\in[n]} |p_i - p|,$$

and treat it as an explicit degradation term (we do not require ϵ_{bias} to be negligible).

The assumptions above formalize computational hardness and the sampler's measurable distributional deviations. Security may degrade in practice if these assumptions are violated or if additional leakage channels are present. In particular, security may degrade if: (i) *side-channel leakage* (timing/power/cache/EM) from the neural sampler or PRNG implementation reveals seeds, internal states, or intermediate activations beyond the partial-state exposure model considered in our security definitions; (ii) *seed reuse* or seed exposure via crash dumps/VM snapshots reduces unpredictability and forward-security guarantees; (iii) *implementation coupling* across generator instances introduces unintended correlations, effectively increasing ϵ_{ind}; or (iv) *model/weight tampering* and deployment drift increase ϵ_{ind} and/or the marginal-rate mismatch (e.g., ϵ_{bias}). These scenarios do not contradict the reduction itself; rather, they enlarge the explicit deviation terms and reduce the concrete security margin.

4.2. The hardness of the GAN-LPN problem

We define the GAN-LPN problem and provide a formal, game-based proof of its computational hardness. The proof is anchored in Assumption 1 (standard LPN hardness) and incorporates explicit degradation terms capturing the non-ideal noise properties of the learned sampler (Assumption 2).

4.2.1. Formal definition and hardness reduction

We begin with a standard lemma used in game-hopping arguments.

Lemma 3. ***(Statistical distance bounds distinguishing gap).*** *For any two distributions D_0, D_1 over a common sample space and any (possibly unbounded) distinguisher $\mathcal{A}$,*

$$\left|\Pr[\mathcal{A}(D_0) = 1] - \Pr[\mathcal{A}(D_1) = 1]\right| \le \Delta(D_0, D_1).$$

Definition 1. (GAN-LPN Problem). The decisional GAN-LPN problem with parameters (m, n, G) is to distinguish between the following two distributions:

$$D_0 : (A,\ A\mathbf{s} \oplus \mathbf{e}) \quad \text{and} \quad D_1 : (A,\ \mathbf{u}),$$

where $A \xleftarrow{\$} \mathbb{F}_2^{n\times m}$, $\mathbf{s} \xleftarrow{\$} \mathbb{F}_2^m$, $\mathbf{u} \xleftarrow{\$} \mathbb{F}_2^n$, and $\mathbf{e} \leftarrow G(\mathbf{z})$ is the generator-induced noise (with $\mathbf{z}$ sampled according to the generator's prescribed sampling mechanism). The advantage of a PPT adversary $\mathcal{A}$ is

$$\mathrm{Adv}_{\mathcal{A}}^{\text{GAN-LPN}}(\lambda) := |\Pr[\mathcal{A}(A, A\mathbf{s} \oplus \mathbf{e}) = 1] - \Pr[\mathcal{A}(A, \mathbf{u}) = 1]|.$$

The hardness of GAN-LPN rests on two pillars: (i) baseline hardness inherited from standard LPN at noise rate δ_1 (Assumption 1), and (ii) a quantified closeness of the generator-induced noise to an independent Bernoulli product distribution, captured by ϵ_{ind} (Assumption 2).

Theorem 3 ((Game-hopping reduction from LPN(δ_1) to GAN-LPN))**.** *Assume Assumption 1 and Assumption 2. Then for any PPT adversary $\mathcal{A}$ against decisional GAN-LPN, there exists a PPT adversary $\mathcal{B}$ against decisional LPN(δ_1) such that*

$$Adv_{\mathcal{A}}^{\text{GAN-LPN}}(\lambda) \le Adv_{\mathcal{B}}^{\mathrm{LPN}(\delta_1)}(\lambda) + \epsilon_{ind}.$$

In particular, if LPN(δ_1) is hard (negligible advantage), then GAN-LPN is hard up to an explicit additive degradation term ϵ_{ind}.

Proof. (via a sequence of games). We define the following games.

Game 0 (Real GAN-LPN). The challenger samples $A \xleftarrow{\$} \mathbb{F}_2^{n\times m}$, $\mathbf{s} \xleftarrow{\$} \mathbb{F}_2^m$, draws $\mathbf{e} \leftarrow G(\mathbf{z})$, and gives $(A, A\mathbf{s} \oplus \mathbf{e})$ to $\mathcal{A}$. In the uniform case it gives $(A, \mathbf{u})$. By definition,

$$\mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game0}}(\lambda) = \mathrm{Adv}_{\mathcal{A}}^{\text{GAN-LPN}}(\lambda).$$

Game 1 (Independent non-identical Bernoulli noise). This game is identical to Game 0 except that the challenger replaces $\mathbf{e}$ with $\tilde{\mathbf{e}}$ sampled independently by coordinates as

$$\tilde{\mathbf{e}} = (\tilde{e}_1, \ldots, \tilde{e}_n), \qquad \tilde{e}_i \xleftarrow{\$} \mathrm{Ber}(p_i) \text{ independently},$$

where $p_i = \Pr[X_i = 1]$ are the generator-induced marginal rates.

By Assumption 2, the statistical distance between the real generator noise and the independent product noise is

$$\Delta\left(P_{\mathbf{e}}, \prod_{i=1}^{n} \mathrm{Ber}(p_i)\right) = \epsilon_{\mathrm{ind}}.$$

Since $(A, A\mathbf{s} \oplus \cdot)$ is a bijection on the noise component for fixed $(A, \mathbf{s})$, the same bound carries to the joint sample distribution. Therefore, by Lemma 1,

$$\left|\mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game0}}(\lambda) - \mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game1}}(\lambda)\right| \le \epsilon_{\mathrm{ind}},$$

and hence

$$\mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game0}}(\lambda) \le \mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game1}}(\lambda) + \epsilon_{\mathrm{ind}}.$$

Game 2 (Standard LPN(δ_1)). We now relate Game 1 to standard LPN(δ_1) via a reduction (no additional statistical loss). Construct a PPT adversary $\mathcal{B}$ for LPN(δ_1) as follows.

$\mathcal{B}$ receives a challenge $(A, \mathbf{b})$ which is drawn either from LPN(δ_1), i.e., $\mathbf{b} = A\mathbf{s} \oplus \mathbf{e}$ with $\mathbf{e} \xleftarrow{\$} \mathrm{Ber}(\delta_1)^n$ (i.i.d.), or from the uniform distribution $(A, \mathbf{u})$.

Define for each i:

$$q_i := \frac{p_i - \delta_1}{1 - 2\delta_1}.$$

Since $p_i \in [\delta_1, \delta_2]$ and $\delta_2 < 1/2$, we have $q_i \in [0, 1/2)$. Sample $\mathbf{r} = (r_1, \ldots, r_n)$ with independent coordinates $r_i \xleftarrow{\$} \mathrm{Ber}(q_i)$, and set

$$\mathbf{b}' := \mathbf{b} \oplus \mathbf{r}.$$

Finally, $\mathcal{B}$ outputs $\mathcal{A}(A, \mathbf{b}')$.

We analyze the two cases.

Uniform case: if $\mathbf{b} = \mathbf{u}$ is uniform, then $\mathbf{b}' = \mathbf{u} \oplus \mathbf{r}$ is also uniform for any fixed $\mathbf{r}$; hence $(A, \mathbf{b}')$ has the uniform distribution.

LPN case: if $\mathbf{b} = A\mathbf{s} \oplus \mathbf{e}$ with $e_i \xleftarrow{\$} \mathrm{Ber}(\delta_1)$ i.i.d., define $\mathbf{e}' := \mathbf{e} \oplus \mathbf{r}$. Then the coordinates remain independent, and for each i,

$$\Pr[e_i' = 1] = \Pr[e_i \oplus r_i = 1] = \delta_1(1 - q_i) + (1 - \delta_1)q_i = \delta_1 + q_i - 2\delta_1 q_i = p_i.$$

Thus $\mathbf{e}'$ is distributed as independent $\mathrm{Ber}(p_i)$ noise, and therefore $(A, \mathbf{b}') = (A, A\mathbf{s} \oplus \mathbf{e}')$ is distributed exactly as the LPN sample used in Game 1.

Consequently,

$$\mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game1}}(\lambda) = \mathrm{Adv}_{\mathcal{B}}^{\mathrm{LPN}(\delta_1)}(\lambda).$$

Combining the above inequalities yields

$$\begin{aligned}\mathrm{Adv}_{\mathcal{A}}^{\text{GAN-LPN}}(\lambda) &= \mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game}\ 0}(\lambda) \\ &\le \mathrm{Adv}_{\mathcal{A}}^{\mathrm{Game}\ 1}(\lambda) + \epsilon_{\mathrm{ind}} \\ &= \mathrm{Adv}_{\mathcal{B}}^{\mathrm{LPN}(\delta_1)}(\lambda) + \epsilon_{\mathrm{ind}}\end{aligned}$$

This completes the proof. □

Corollary 3. ***(Optional comparison to i.i.d.*** $\mathrm{Ber}(p)^n$***).*** *If a target rate p is fixed and $\epsilon_{bias} := \max_i |p_i - p|$, then*

$$\Delta\left(\prod_{i=1}^{n} \mathrm{Ber}(p_i),\ \mathrm{Ber}(p)^n\right) \le \sum_{i=1}^{n} |p_i - p| \le n \cdot \epsilon_{bias}.$$

Hence one may additionally relate Game 1 to an i.i.d. LPN(p) distribution with an explicit additive loss of at most $n\epsilon_{bias}$ (and ϵ_{ind} from Game 0 to Game 1).

4.2.2. Analysis of unexpected vulnerabilities and hybrid adversaries

A paramount concern when modifying a foundational cryptographic primitive is the potential introduction of unforeseen attack vectors. We justify the security of our GAN-LPN variant by showing that its deviation from the standard LPN noise model is *controlled and explicitly bounded*, and therefore appears only as additive degradation terms in the reduction. Consequently, the construction preserves the security properties of standard LPN, including resilience against hybrid classical-quantum adversaries.

Theoretical capacity for small bias. As established in Section 3.2.4 (Theorem 2 and Lemma 1), our neural generator architecture has sufficient expressive power to approximate the target Bernoulli noise distribution, yielding a (potentially negligible) approximation error $\epsilon_G(\lambda)$.

Implications for security. Our security bounds do not require $\epsilon_G(\lambda)$ (nor per-coordinate marginal bias) to be negligible. Instead, deviations from the ideal i.i.d. LPN noise model are quantified explicitly by ϵ_{ind} (departure from product structure) together with the marginal-rate mismatch term (e.g., $n\epsilon_{\mathrm{bias}}$ when aligning $\prod_i \mathrm{Ber}(p_i)$ to $\mathrm{Ber}(p)^n$). These quantities enter the proof as explicit additive losses. Concretely, any non-negligible advantage against GAN-LPN implies a non-negligible advantage against standard LPN, up to an additive loss bounded by $\epsilon_{\mathrm{ind}} + n\epsilon_{\mathrm{bias}}(+n\epsilon_G)$.

Resilience against hybrid classical-quantum adversaries. The reduction in Theorem 3 is black-box and holds for any PPT adversary, including those with quantum computational power. Thus, under the standard assumption that LPN is post-quantum hard for the selected parameters, the GAN-LPN problem provides equivalent security guarantees, modulo the explicit deviation terms above.

4.3. Security analysis of the PRNG

This section provides a unified security analysis of the proposed PRNG under the adversary model in Section 4.1. We analyze (i) pseudorandomness (computational indistinguishability from uniform), (ii) forward security under partial state exposure, and (iii) seed irrecoverability.

4.3.1. Pseudorandomness analysis

The pseudorandomness of the output sequence is the cornerstone security property of our PRNG, signifying that its output is computationally indistinguishable from a truly random sequence to any efficient adversary. This property is not merely a statistical guarantee but a cryptographic one, and in our analysis it is reduced to the hardness of the induced GAN-LPN problem established from Assumption 1 via Theorem 3.

Theorem 4 (Pseudorandomness)**.**

Let $q = q(\lambda)$ be the number of PRNG iterations output to the adversary. Assume Assumption 1 holds. By Theorem 3, the induced GAN-LPN distribution inherits hardness from standard decisional LPN at noise rate δ_1. Assume further that in each iteration the GAN-based sampler outputs a noise vector $\mathbf{e}_t$ whose distribution D_e satisfies

$$\Delta(D_e, \mathrm{Ber}(p)^n) \ \le\ \epsilon_{noise},$$

where

$$\epsilon_{noise} := \epsilon_{ind} + n\epsilon_{bias} \ \ (\text{and optionally } + n\epsilon_G \text{ for finite-precision effects}).$$

Here p serves only as a target reference rate used to quantify the mismatch of the learned sampler, whereas the underlying hardness baseline remains the standard LPN assumption at rate δ_1. If the fixed matrix $M \in \mathbb{F}_2^{m\times n}$ is sampled uniformly at setup and kept confidential, then the PRNG output sequence

$\{z_t\}_{t=1}^{q}$ is computationally indistinguishable from uniform. In particular, for any PPT adversary $\mathcal{A}$,

$$\left|\Pr[\mathcal{A}(\{z_t\}_{t=1}^{q})=1]-\Pr[\mathcal{A}(U)=1]\right| \le \mathrm{Adv}^{\mathrm{dist}}_{\text{GAN-LPN}}(\mathcal{B}) + q\cdot\epsilon_{\text{noise}}+\mathrm{negl}(\lambda) \le \mathrm{Adv}^{\mathrm{dist}}_{\mathrm{LPN}(\delta_1)}(\mathcal{B}')+q\cdot\epsilon_{\text{noise}}+\mathrm{negl}(\lambda),$$

for some PPT distinguishers $\mathcal{B},\mathcal{B}'$ constructed from $\mathcal{A}$.

Proof. (hybrid argument). We define three hybrids Hyb_0, Hyb_1, Hyb_2 as follows:

(1) Hyb_0 (Real PRNG): In each iteration, the noise vector $\mathbf{e}_t \leftarrow G(r_t)$ is generated by the GAN-based sampler, and $\mathbf{c}_t = M^\top v_t + \mathbf{e}_t$ is computed as in the real construction (and z_t is derived from $\mathbf{c}_t$ exactly as specified by the PRNG).

(2) Hyb_1 (Ideal reference noise): Replace each $\mathbf{e}_t$ with $\tilde{\mathbf{e}}_t \leftarrow \mathrm{Ber}(p)^n$, keeping all other components unchanged.

(3) Hyb_2 (Uniform outputs): Replace each $\mathbf{c}_t$ with $\tilde{\mathbf{c}}_t \xleftarrow{\$} \mathbb{F}_2^n$ (and keep the same output derivation $z_t \leftarrow \mathsf{Out}(\tilde{\mathbf{c}}_t)$).

We bound the distinguishing advantage between consecutive hybrids.

Step 1: $Hyb_0 \approx Hyb_1$. By assumption, the statistical distance between the real sampler output distribution and the ideal Bernoulli reference distribution satisfies

$$\Delta(\mathcal{D}_e, \mathrm{Ber}(p)^n) \le \epsilon_{\text{noise}}.$$

Since the PRNG runs for q iterations and the remainder of the computation is deterministic given the sampled noise, a standard hybrid/telescoping argument yields

$$\Delta(Hyb_0, Hyb_1) \;\le\; q\cdot\epsilon_{\text{noise}}.$$

Equivalently, for any PPT $\mathcal{A}$,

$$\left|\Pr[\mathcal{A}(Hyb_0)=1]-\Pr[\mathcal{A}(Hyb_1)=1]\right| \;\le\; q\cdot\epsilon_{\text{noise}}.$$

Step 2: $Hyb_1 \approx Hyb_2$. In Hyb_1, each $\mathbf{c}_t = M^\top v_t + \tilde{\mathbf{e}}_t$ is formed using ideal product Bernoulli noise $\tilde{\mathbf{e}}_t \leftarrow \mathrm{Ber}(p)^n$. This can be viewed as an idealized special case of the GAN-LPN distribution class introduced in Definition 2, in which the coordinate rates are all equal to the reference value p and the product structure is exact. Suppose there exists a PPT adversary $\mathcal{A}$ that distinguishes Hyb_1 from Hyb_2 with advantage $\alpha(\lambda)$. We build a PPT distinguisher $\mathcal{B}$ for the GAN-LPN distinguishing problem as follows: $\mathcal{B}$ receives challenge samples that are either distributed according to the relevant GAN-LPN instance or uniformly random over $\mathbb{F}_2^n$. It uses these challenge samples to simulate the PRNG outputs to $\mathcal{A}$ by setting the PRNG's per-iteration $\mathbf{c}_t$ to the provided challenge and deriving z_t exactly as in the real scheme. If $\mathcal{A}$ outputs 1 (meaning "looks like Hyb_1"), $\mathcal{B}$ outputs 1; otherwise it outputs 0. Then $\mathcal{B}$'s advantage is exactly $\alpha(\lambda)$. Hence,

$$\left|\Pr[\mathcal{A}(Hyb_1)=1]-\Pr[\mathcal{A}(Hyb_2)=1]\right| \le \mathrm{Adv}^{\mathrm{dist}}_{\text{GAN-LPN}}(\mathcal{B})+\mathrm{negl}(\lambda) \le \mathrm{Adv}^{\mathrm{dist}}_{\mathrm{LPN}(\delta_1)}(\mathcal{B}')+\mathrm{negl}(\lambda),$$

where the second inequality follows from Theorem 3.

Combining Step 1 and Step 2 by the triangle inequality completes the proof:

$$\left|\Pr[\mathcal{A}(Hyb_0)=1]-\Pr[\mathcal{A}(Hyb_2)=1]\right| \le \mathrm{Adv}^{\mathrm{dist}}_{\text{GAN-LPN}}(\mathcal{B}) + q\cdot\epsilon_{\text{noise}}+\mathrm{negl}(\lambda) \le \mathrm{Adv}^{\mathrm{dist}}_{\mathrm{LPN}(\delta_1)}(\mathcal{B}')+q\cdot\epsilon_{\text{noise}}+\mathrm{negl}(\lambda).$$

Since Hyb_2 is uniform by definition, the output $\{z_t\}_{t=1}^{q}$ is computationally pseudorandom up to the stated bound. □

4.3.2. Forward security analysis

We formalize *forward security* as *backtracking resistance*: even if the adversary compromises the current internal state w_t, it should not be able to recover any past outputs $z_{t'}$ for $t' < t$ with non-negligible advantage.

Theorem 5. ***(Forward Security / Backtracking Resistance).*** *Let $q = q(\lambda)$ be the number of iterations. Assume: (i) **State erasure:** after computing w_{i+1}, the PRNG securely erases w_i and all intermediate values not included in w_{i+1}; (ii) **One-way state update:** the state update function Upd is one-way, i.e., given $w_{i+1} = \mathsf{Upd}(w_i, z_i)$ it is computationally infeasible for any PPT adversary to recover w_i (or any value that allows computing z_i) with non-negligible probability; (iii) **Pseudorandomness:** the output sequence is pseudorandom as established in Theorem 4.*

Then for any PPT adversary $\mathcal{A}$ given the current state w_t,

$$\Pr\left[\mathcal{A}(w_t) \text{ outputs any } z_{t'} \text{ for some } t' < t\right] \le 2^{-|z|} + \mathrm{Adv}^{\mathrm{ow}}_{\mathsf{Upd}}(\mathcal{B}_1) + \mathrm{Adv}^{\mathrm{pr}}_{\mathrm{PRNG}}(\mathcal{B}_2),$$

where $|z|$ is the output length per iteration, and $\mathcal{B}_1, \mathcal{B}_2$ are PPT reductions. In particular, by Theorem 4, the success probability is negligible under the above assumptions.

Proof. Fix any $t' < t$. Consider the following two experiments:

Real: generate PRNG outputs $(z_1, \ldots, z_t)$ and states $(w_1, \ldots, w_t)$ honestly, then give w_t to $\mathcal{A}$.

Ideal: generate the same state evolution, but replace the past output $z_{t'}$ with an independent uniform string $u \leftarrow \{0,1\}^{|z|}$, and give w_t to $\mathcal{A}$.

We show that $\mathcal{A}$ cannot distinguish these experiments with non-negligible advantage and thus cannot recover $z_{t'}$.

First, by the **state erasure** and **one-wayness** of Upd, the current state w_t does not allow efficient reconstruction of any previous state $w_{t'}$ (nor any information sufficient to recompute $z_{t'}$) except with probability bounded by $\mathrm{Adv}^{\mathrm{ow}}_{\mathsf{Upd}}(\mathcal{B}_1)$.

Second, conditioned on not inverting the update, the only remaining way to predict $z_{t'}$ from w_t is to distinguish the real PRNG outputs from uniform, which is exactly bounded by the pseudorandomness advantage established in Theorem 4. Therefore,

$$\left|\Pr[\mathcal{A} \text{ wins in Real}] - \Pr[\mathcal{A} \text{ wins in Ideal}]\right| \le \mathrm{Adv}^{\mathrm{ow}}_{\mathsf{Upd}}(\mathcal{B}_1) + \mathrm{Adv}^{\mathrm{pr}}_{\mathrm{PRNG}}(\mathcal{B}_2).$$

In the Ideal experiment, $z_{t'}$ is uniform and independent of $\mathcal{A}$'s view, hence

$$\Pr[\mathcal{A} \text{ outputs } z_{t'}] \le 2^{-|z|}.$$

Combining the above inequalities proves the claim. □

4.3.3. Seed irrecoverability analysis

We formalize *seed irrecoverability* as the infeasibility of extracting the initial seed from the PRNG outputs. Concretely, given the output transcript $(z_1, \ldots, z_q)$ for $q = q(\lambda)$ iterations, an adversary should not be able to output the exact initial PRNG seed with non-negligible probability.

Theorem 6. (Seed Irrecoverability). *Let $\mathsf{PRNG}(\mathsf{seed})$ denote the deterministic output generation algorithm of the proposed construction, where all public parameters are fixed and the matrix M is fixed after setup. Assume the PRNG is pseudorandom as established in Theorem 4. Then, for any PPT adversary $\mathcal{A}$,*

$$\Pr\left[\mathcal{A}(z_1, \ldots, z_q) = \mathsf{seed}\right] \;\le\; 2^{-|\mathsf{seed}|} \;+\; \mathrm{Adv}^{\mathrm{pr}}_{\mathrm{PRNG}}(\lambda).$$

In particular, by Theorem 4 and Theorem 3, the success probability is negligible under Assumption 1 and the explicit noise-deviation bounds.

Proof. We consider two experiments:

Real: sample $\mathsf{seed} \leftarrow \{0,1\}^{m+\ell}$, run the PRNG to obtain $(z_1, \ldots, z_q) \leftarrow \mathsf{PRNG}(\mathsf{seed})$, and give the transcript to $\mathcal{A}$.

Ideal: sample an independent uniform transcript $U \leftarrow (\{0,1\}^{|z|})^q$ and give U to $\mathcal{A}$.

Let $\mathcal{A}$ output a candidate seed seed'. Define an event E that seed' equals the true seed in the Real experiment. We bound $\Pr[E]$.

First, in the Ideal experiment, the transcript is independent of the true seed. Therefore, for any (possibly randomized) algorithm,

$$\Pr[\mathcal{A}(U) = \mathsf{seed}] \le 2^{-|\mathsf{seed}|}.$$

Second, if $\mathcal{A}$ achieves a noticeably higher success probability in the Real experiment than in the Ideal experiment, then we can build a distinguisher D for PRNG pseudorandomness: on input a transcript T, run $\mathcal{A}(T)$ to get seed', then locally recompute $T' = \mathsf{PRNG}(\mathsf{seed}')$ (using the same fixed parameters and the fixed matrix M embedded in D) and output 1 iff $T' = T$. In the Real case, D outputs 1 with probability $\Pr[E]$; in the Ideal case, it outputs 1 with probability at most $2^{-|\mathsf{seed}|}$. Hence,

$$\Pr[E] - 2^{-|\mathsf{seed}|} \le \mathrm{Adv}^{\mathrm{pr}}_{\mathrm{PRNG}}(\lambda).$$

Rearranging yields the claimed bound:

$$\Pr[E] \le 2^{-|\mathsf{seed}|} + \mathrm{Adv}^{\mathrm{pr}}_{\mathrm{PRNG}}(\lambda).$$

Finally, invoking Theorem 4 reduces the bound to Assumption 1 together with the explicit noise-deviation terms. □

5. Instance of the pseudorandom number generator

This section presents a comprehensive empirical evaluation of the proposed GAN-LPN PRNG, providing empirical evidence on statistical quality and practical behavior. The evaluation is structured to provide a multi-faceted assessment, beginning with the concrete instantiation of the generator and progressing through statistical testing, security analysis under practical adversarial scenarios, and performance benchmarking. The selected benchmarks encompass a range of predominant design paradigms, including: the AES-CTR PRNG[39] based on block cipher design; the ChaCha20[22] and HC-128[21] algorithms utilizing stream cipher constructions; the BBS [19] PRNG relying on the hardness of large integer factorization; the MT[20] algorithm; a GAN-based PRNG[10]; and a LPN-based PRNG[37].

The GAN-LPN PRNG is implemented using PaddlePaddle. As a deep learning framework analogous to PyTorch, PaddlePaddle is characterized by its high efficiency, flexibility, and comprehensive functionality, which enable the implementation and optimization of complex models. The hardware configuration of the PaddlePaddle platform employed is as follows: the CPU is an Intel(R) Xeon(R) Platinum 8350 CPU operating at 2.60 GHz with 2 cores, accompanied by 8 GB of random access memory; the GPU is a Tesla V100 with 32 GB of video random access memory. For the software configuration, Python 3.10.10 and PaddlePaddle 2.6.2 are adopted. Relevant code can be accessed via the URL https://gitee.com/guangandy/ganlpnprng.

5.1. Security parameterization

To concretely justify our 128-bit security target, we estimate the bit-security of the underlying LPN instance using the public LPN Estimator [40]. The tool is designed for concrete-security evaluation of LPN/dual-LPN and implements multiple state-of-the-art attack families, including Pooled Gauss (Gauss) [41], Statistical Decoding (SD) [42], Statistical Decoding 2.0 (SD2) [43], Stern-Dumer SD (SDISD) [44], and Becker-Joux-May-Meurer SD (BJMMISD) [45]. For transparency and reproducibility, we report the estimator input parameters and the minimum work-factor among the implemented attacks, since the overall security is determined by the best known attack.

For the concrete instance used in this paper, we take $(n, m, h) = (1024, 320, 200)$, where n is the output/noise dimension, m is the secret/state dimension, and h is the Hamming weight of the noise vector in the estimator setting. Equivalently, this corresponds to an effective noise rate $p = h/n = 200/1024 \approx 0.1953$. Accordingly, we ran the estimator on the parameter set $(1024, 320, 200)$. The resulting log(2) time costs (in bits) for the implemented attacks are summarized in Table 2. Following the standard best-attack principle, we take the minimum over all reported attacks as the concrete security level:

$$\mathrm{Sec}(n, m, h) := \min_{\mathrm{Attack}} \log_2(\mathrm{Cost}_{\mathrm{Attack}}(n, m, h)).$$

For this instance, the minimum estimated work-factor is 128.89 bits (BJMM), yielding an estimated attack cost of at least $2^{128.89}$ operations.

Table 2
Concrete security estimates for Exact LPN with $(1024, 320, 200)$ using the LPN Estimator. We report $\log_2$ time costs and take the minimum as the final security level.

Attack family	Estimated time cost (bits)
Gauss	146.65
SD	292.71
SD2	145.03
SDISD	133.00
BJMMISD	128.89
Minimum (best known attack)	**128.89**

Table 3
Core parameters of the Instance.

Parameter	Value	Description
n	1024	Output vector dimension
m	320	Secret state dimension
p	0.2	Target noise rate used in training
ℓ	128	Latent seed dimension

Therefore, this parameter set meets the 128-bit classical security target against the best known attacks covered by the estimator.

Finally, we emphasize how this parameterization interacts with our GAN-based sampler. The work-factor above quantifies the hardness of the ideal LPN core determined by (n,m,p). Any deviation of the learned sampler from ideal Bernoulli noise is handled separately in our proofs via explicit degradation terms (e.g., ϵ_{ind} and $n\epsilon_{\mathrm{bias}}$), which enter the reductions additively and do not change the estimator's cryptanalytic evaluation of the chosen (n,m,p) instance.

The core parameters used in this instantiation are summarized in Table 3.

5.2. GAN-LPN Training and experimental validation

We implement the Bernoulli Noise Expansion Sampler using a WGAN-GP, whose purpose is to approximate the noise distribution required by the underlying LPN-based pseudorandom generator. The core parameters of the instantiated GAN-LPN system are summarized in Table 3. In particular, the sampler expands a uniformly distributed latent seed of length $\ell = 128$ into an output noise vector of dimension $n = 1024$, with a training target corresponding to a Bernoulli distribution of rate $p = .2$. For the concrete security parameterization in Section 5.1, we use the instantiated estimator setting $(n, m, h) = (1024, 320, 200)$, which corresponds to an effective rate $p = 200/1024 \approx 0.1953$.

The generator internally produces continuous activations, which are binarized at the output layer via BinarySTE/sign during forward propagation. Each output coordinate is produced by an independent fully connected subnetwork, and no parameters are shared across these subnetworks. This design is intended to promote statistical independence among output bits, which is important for preserving the intended hardness properties of the underlying LPN-style construction. Each subnetwork consists of multiple dense layers with layer normalization applied to intermediate layers to improve numerical stability and mitigate gradient explosion or vanishing. Hardtanh activation functions are employed to constrain intermediate activations within a bounded range, facilitating controlled approximation of Bernoulli-distributed outputs.

The final output layer adopts BinarySTE. During forward propagation, a sign function is applied to generate binary outputs, while during backpropagation, gradients are approximated using the derivative of the sigmoid function. This surrogate-gradient strategy enables effective end-to-end training despite the discrete nature of the output space.

The discriminator is designed to distinguish between sequences generated by the generator and genuine noise samples drawn from an i.i.d. Bernoulli distribution $\mathrm{Ber}(p)^n$ with $p = .2$. It receives a 1024-dimensional

real-valued input sequence and first processes it through a shared feature extraction module composed of three fully connected layers: Linear(1024, 2048), followed by LeakyReLU, and then Linear(2048, 1024).

The extracted high-level features are subsequently fed into two specialized evaluation heads. The first head estimates the empirical mean of the input sequence using a structure Linear(1024, 512)-LeakyReLU-Linear(512, 1)-Tanh, thereby assessing alignment with the target Bernoulli bias $p = .2$. The second head evaluates second-order statistical dependence via a covariance-sensitive pathway Linear(1024, 256)-LeakyReLU-Linear(256, 1)-Tanh, which penalizes inter-bit correlations that would weaken the effective LPN noise model.

Training is performed with a batch size of 1024. The learning rates for the generator and discriminator are set to 2×10^{-4} and 4×10^{-4}, respectively. Real samples are generated synthetically by sampling from $\mathrm{Ber}(0.2)^{1024}$. The model is trained for a total of 1000 epochs. In each epoch, the discriminator is updated three times per batch, while the generator is updated once. The generator output is binarized such that values greater than zero are mapped to 1, and values less than or equal to zero are mapped to 0.

Fig. 5 illustrates the discriminator loss, empirical mean, and covariance of the generated samples throughout the training process. During the initial phase of training, the discriminator loss decreases steadily as the critic learns to distinguish generated samples from true Bernoulli noise. Around epoch ≈ 550, a noticeable transition occurs, which is characteristic of WGAN-GP training when the generator output begins to closely approximate the target distribution. This transition reflects a redistribution of gradient pressure rather than instability, indicating that the discriminator can no longer trivially separate real and generated samples.

Concurrently, the empirical mean of the generated sequences decreases from approximately 0.5 toward the target value of $p = .2$ and stabilizes thereafter, demonstrating that the generator successfully learns the intended Bernoulli bias. The covariance metric initially increases during early training due to random initialization effects but gradually decays and stabilizes at a low level, indicating that undesired inter-bit dependencies are effectively suppressed.

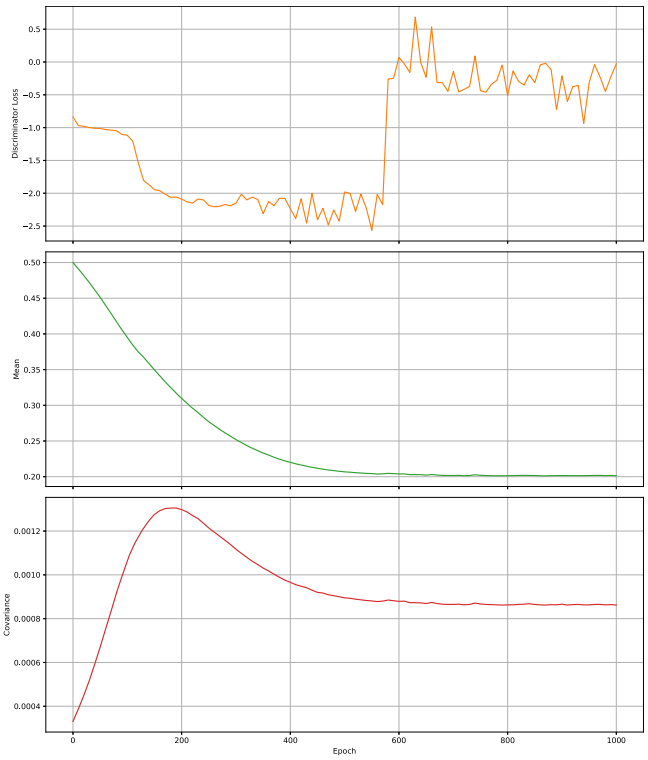

Fig. 5. Metrics during the GAN training process: Discriminator Loss, Mean value of the generated data, Covariance value of the generated data.

5.3. Statistical randomness validation

In this section, we instantiate a secure PRNG based on the GAN-LPN assumption. The design incorporates concrete parameter selection, model training procedures, and a comprehensive evaluation of the statistical and cryptographic properties of its output sequence using the NIST SP 800-22 randomness test suite [23] and Chinese standard GM/T 0005–2021 Randomness Tests [24].

In the experiments, the proposed PRNG generated a total of 1000 binary sequences, each consisting of 1,048,576 bits. This configuration adheres to the standard requirements for randomness testing and ensures sufficient statistical significance for evaluating the randomness of the generated data.

5.3.1. NIST SP 800-22 Randomness testing

This study first employs the NIST SP 800-22, a statistical test suite for randomness evaluation, to systematically assess the randomness properties of the generated binary sequences. This test suite is widely recognized in the field of cryptography as a standard for evaluating the performance of random number generators. It comprises 15 complementary statistical tests, including the Frequency Test, Block Frequency Test, Runs Test, Discrete Fourier Transform Test, and the Random Excursions Test, among others. Notably, certain tests such as Random Excursions consist of multiple subtests.

The significance level is set to $\alpha = 0.01$, meaning that if the p-value of any individual test falls below this threshold, the test is considered to have failed. This level corresponds to a 99% confidence interval and provides a relatively stringent criterion for statistical inference, thereby helping to reduce the probability of Type I errors and enhancing the reliability and stability of the test results.

Table 4
NIST Tests results.

Test	Min P-value	MinPassCount	PassThreshold	Passed
Frequency	0.254411	987	980	Yes
BlockFrequency	0.829047	993	980	Yes
CumulativeSums	0.057510	987	980	Yes
Runs	0.183547	986	980	Yes
LongestRun	0.532132	994	980	Yes
Rank	0.765632	988	980	Yes
FFT	0.090936	990	993	Yes
NonOverlappingTemplate	0.010457	985	980	Yes
OverlappingTemplate	0.651693	992	980	Yes
Universal	0.492436	989	980	Yes
ApproximateEntropy	0.605916	987	980	Yes
RandomExcursions	0.014981	615	614	Yes
RandomExcursionsVariant	0.014981	615	614	Yes
Serial	0.737915	989	980	Yes
LinearComplexity	0.759756	990	980	Yes

The experimental results are summarized in Table 4, which lists the minimum p-values and the lowest pass counts across all 15 main tests and their respective subtests. The Random Excursions and its variant tests include multiple subtests, among which the smallest p-value observed is 0.014981. The corresponding passing proportions are 615 out of 628, exceeding the required threshold of 614 out of 628. For all other tests, the minimum p-values are greater than 0.01, and the number of passed sequences exceeds the threshold of 980 out of 1000. Each test reports minimum p-values that do not indicate significant deviations from randomness, and the proportion of passed sequences consistently meets or exceeds the required thresholds. These results collectively confirm the high quality of randomness exhibited by the generated sequences. The proposed PRNG demonstrates robust performance across all 15 tests of the NIST SP 800-22 suite.

Table 5
GM/T 0005–2021 Tests results.

Test	s_{pass}	P_T	Passed
Frequency	992	0.356182	Yes
Block Frequency	987	0.585209	Yes
Poker	983	0.770499	Yes
Overlapping Subsequence	987	0.229559	Yes
Runs	996	0.125927	Yes
Run Distribution	994	0.801865	Yes
LongestRun	990	0.778227	Yes
Binary Derivation	984	0.189625	Yes
Autocorrelation	990	0.735908	Yes
Rank	990	0.549331	Yes
CumulativeSums	995	0.418120	Yes
ApproximateEntropy	995	0.278461	Yes
LinearComplexity	986	0.331408	Yes
Maurer General Statistics	988	0.550347	Yes
FFT	990	0.938463	Yes

5.3.2. Chinese standard GM/T 0005–2021 Randomness Tests

The GM/T 0005–2021 Randomness Test Specification, issued by the National Cryptography Administration of China, serves as a key technical standard for evaluating the security and performance of PRNGs. It defines a set of 15 complementary statistical tests specifically designed to assess the randomness of binary sequences in cryptographic contexts. In comparison with the NIST SP 800-22 test suite, GM/T 0005–2021 replaces certain tests such as the Random Walk Test and its variant, along with the Non-overlapping and Overlapping Template Matching Tests, with alternative methods including the Poker Test, Run Distribution Test, Binary Derivative Test, and Autocorrelation Test, which are considered more suitable for cryptographic evaluation under Chinese standards.

Each test is conducted on binary sequences of length 1,000,000 bits, with a significance level of $\alpha = 0.01$, ensuring a balance between computational efficiency and statistical rigor. The assessment of passing results is based on two statistical criteria that must both be satisfied for each test. First, the proportion of samples whose p-values are greater than or equal to α must meet a minimum threshold. Specifically, given a total of $s = 1000$ sample sequences, the number of passing samples s_{pass} must satisfy the inequality:

$$s_{\text{pass}} \geq s\left(1 - \alpha - 3\sqrt{\frac{\alpha(1-\alpha)}{s}}\right),$$

which corresponds to at least 981 successful samples out of 1000. Second, the distribution of Q-values across all samples must approximate a uniform distribution over the interval $[0, 1]$. A uniformity metric P_T is calculated for this purpose, and the test is considered passed if P_T is no less than the uniformity significance threshold $x_T = 0.0001$.

Only when all 15 tests collectively satisfy both the sample pass rate requirement and the uniformity condition can a sequence be certified as having passed the GM/T 0005–2021 randomness test. In this study, a conservative approach was adopted by reporting the minimum values of s_{pass} and P_T observed across all tests, as summarized in Table 5. The results demonstrate that all tests exceeded the required minimum pass count of 981, and all P_T values were above 0.0001. These findings confirm that the proposed PRNG successfully passes the full set of 15 GM/T 0005–2021 tests.

5.3.3. Autocorrelation (ACF) analysis

In addition to standard statistical randomness tests, we evaluate the temporal independence of PRNG outputs via the sample autocorrelation function. Autocorrelation analysis is a widely used diagnostic to detect unintended linear dependencies between output bits at different time lags, which may otherwise remain unnoticed by frequency-based tests.

Given a generated binary sequence $\{x_t\}_{t=1}^{N}$, the sample autocorrelation at lag k is computed using the standard normalized estimator after centering. Under the ideal i.i.d. bit model, the expected autocorrelation is zero for all non-zero lags, and any observed deviation is attributed to finite-sample noise. Following common practice, a lag is considered statistically significant if $|\rho(k)|$ exceeds the 95% confidence bound.

Table 6 reports summary ACF statistics for all evaluated PRNGs, including the maximum absolute autocorrelation over the tested lag window, the proportion of statistically significant lags, and the mean and standard deviation of $|\rho(k)|$. All generators exhibit very small autocorrelation values on the order of 10^{-3}, with *zero* statistically significant lags observed in all cases.

Notably, the proposed GAN-LPN PRNG achieves a maximum absolute autocorrelation of 0.001252, with mean and standard deviation comparable to those of well-established cryptographic PRNGs such as AES-CTR-DRBG, ChaCha20, and HC-128. These results indicate that incorporating the GAN-based noise sampler does not introduce detectable linear temporal dependencies into the output bitstream.

5.3.4. Non-linear dependency analysis via mutual information

While autocorrelation analysis captures linear temporal dependencies, it is insufficient to detect more general non-linear relationships between output bits. To address this concern, we further evaluate the presence of non-linear dependencies in PRNG outputs using mutual information (MI), a widely adopted information-theoretic measure capable of detecting arbitrary statistical dependence.

Given a binary output sequence $\{x_t\}_{t=1}^{N}$, we estimate the mutual information $I(x_t; x_{t+k})$ between bits separated by lag k. For an ideal i.i.d. source, the mutual information is zero for all $k > 0$. In practice, finite-sample effects lead to small non-zero estimates, and thus both the magnitude and distribution of MI values across lags are considered. Following conservative practice, we report the maximum MI, the mean and standard deviation of MI over the tested lag window, as well as the proportion of lags whose MI exceeds a small threshold (10^{-3} bits), which would indicate non-negligible dependency.

Table 7 summarizes the MI-based non-linear dependency statistics for all evaluated PRNGs. Across all schemes, the maximum observed MI values are on the order of 10^{-5} bits, while the mean MI is effectively zero and no lag exhibits MI exceeding the predefined threshold. In particular, the proposed GAN-LPN PRNG achieves a maximum MI of 9×10^{-6} bits, with dispersion comparable to that of AES-CTR-DRBG, ChaCha20, and HC-128.

These results indicate that the proposed construction does not introduce detectable non-linear temporal dependencies in the output bitstream. Combined with the autocorrelation analysis in Section 5.3.3, the MI evaluation confirms that both linear and non-linear forms of statistical dependence remain indistinguishable from those of high-quality cryptographic PRNGs within the sensitivity of the experiment.

5.4. Differential analysis: Hamming distance (HD) test

Differential cryptanalysis is a powerful attack paradigm that infers critical information about a system's internal state by observing the propagation of small changes from the input to the output. In the context of PRNGs, a pertinent threat model is the seed differential attack. In this scenario, an adversary, potentially via fault injection, induces a single-bit flip in the initial seed. By comparing the original output sequence with the sequence generated from the perturbed seed, the attacker aims to discover statistical correlations that could compromise the PRNG's security. The fundamental property required to resist such attacks is a strong avalanche effect, whereby a minimal alteration in the seed provokes extensive and unpredictable changes in the output sequence, rendering the two outputs computationally indistinguishable.

To quantitatively evaluate the resistance of our proposed scheme to differential attacks, we employ the normalized mean Hamming distance as the primary metric for the avalanche effect. For two seeds, S and S', differing by a single bit, and their corresponding output sequences O and O' of length L, the Hamming distance $HD(O, O')$ is defined as the

Table 6
Autocorrelation (ACF) comparison of PRNG output bitstreams.

Metric	AES-CTR	BBS	ChaCha20	GAN-LPN	GAN	HC-128	LPN	MT19937	Ideal
Max $\|\rho(k)\|$	0.001275	0.001337	0.001248	0.001252	0.001130	0.001186	0.001190	0.001327	0
Significant lag ratio	0.000%	0.000%	0.000%	0.000%	0.000%	0.000%	0.000%	0.000%	0%
Mean $\|\rho(k)\|$	0.000251	0.000255	0.000254	0.000257	0.000257	0.000252	0.000253	0.000247	0
Std. of $\|\rho(k)\|$	0.000189	0.000193	0.000192	0.000194	0.000193	0.000193	0.000194	0.000188	small

Table 7
Mutual information (MI) comparison for detecting non-linear dependencies in PRNG outputs.

Metric	AES-CTR	BBS	ChaCha20	GAN-LPN	GAN	HC-128	LPN	MT19937	Ideal
Max MI (bits)	1.0×10^{-5}	1.1×10^{-5}	0.8×10^{-5}	0.9×10^{-5}	1.0×10^{-5}	0.9×10^{-5}	0.8×10^{-5}	1.0×10^{-5}	0
Mean MI (bits)	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	0
Std. of MI (bits)	1.0×10^{-6}	1.0×10^{-6}	1.0×10^{-6}	1.0×10^{-6}	1.0×10^{-6}	$< 10^{-6}$	1.0×10^{-6}	1.0×10^{-6}	small
MI $> 10^{-3}$ lag ratio	0.000%	0.000%	0.000%	0.000%	0.000%	0.000%	0.000%	0.000%	0%

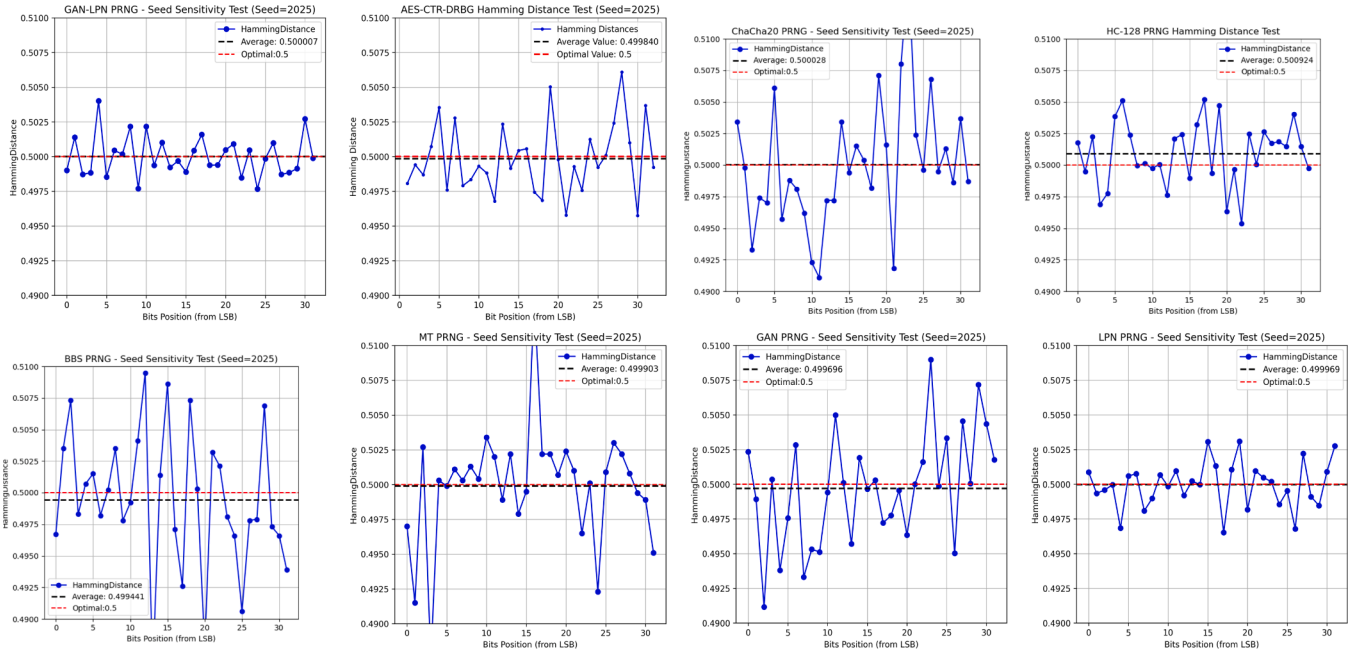

Fig. 6. Hamming distance of the comparing Prngs.

number of differing bits. The normalized differential rate DR is then given by the formula:

$$DR = \frac{HD(O, O')}{L}$$

For an ideal cryptographically secure PRNG, the expected value of DR should converge to 0.5, indicating that the change in the output is complete and random.

The experimental analysis involved fixing the initial seed to 2025 and generating 32 pairs of seeds, where each pair differed by exactly one bit that was flipped sequentially from the least significant bit to the most significant bit. For each pair, output sequences of 10,000 bits were generated, and the normalized differential rate DR was computed.

As illustrated in Fig. 6, the proposed scheme achieved a mean differential rate of $\overline{DR} = 0.500729$, which is very close to the theoretical optimum of 0.5. Furthermore, it demonstrated the standard deviation is very small.

5.5. Adversarial robustness via retrained learned distinguishers

We evaluate our GAN-LPN PRNG under a learned distinguisher attack[40]. The experiment models a strong polynomial-time adversary that obtains a large collection of PRNG outputs and trains an independent discriminator D' from scratch to distinguish PRNG outputs from ideal randomness. A PRNG is considered computationally indistinguishable if no such efficient distinguisher can achieve a non-negligible advantage over random guessing.

Data preparation. For each PRNG, we generate a long binary output file and lazily load samples as fixed-length bit sequences. Each sample is labeled as either Real (PRNG output) or Ideal (uniform random bits of the same length generated by a cryptographically secure source). We split the dataset into training/validation/test sets using a fixed `SEED` to ensure reproducibility. The number of samples per class is controlled by `SAMPLES_PER_CLASS` and `MAX_SAMPLES_FOR_LENGTH`.

Sequence lengths and downsampling. We evaluate three output lengths $L \in \{2^{14}, 2^{16}, 2^{20}\}$. If $L >$ `MAX_PROCESSABLE_LENGTH`, we downsample the bit sequence by a factor of $\lceil L/$`MAX_PROCESSABLE_LENGTH`$\rceil$ and feed the resulting `effective_length` into the models. For the Transformer-based distinguisher, if `effective_length` exceeds `TRANSFORMER_MAX_SEQ`, the run is skipped to avoid exceeding memory limits.

Adversary models (D'). We consider three architectures to capture different dependency patterns: (i) a 1D-CNN (sensitive to local motifs

Table 8
Learned distinguisher attack with retrained D'.

PRNG	$L = 2^{14}$		$L = 2^{16}$		$L = 2^{20}$	
	max $\widehat{\text{Adv}}$ / AUC	p	max $\widehat{\text{Adv}}$ / AUC	p	max $\widehat{\text{Adv}}$ / AUC	p
GAN-LPN PRNG	0.012 / 0.506	0.29	0.016 / 0.508	0.21	0.022 / 0.511	0.14
AES-CTR-DRBG	0.006 / 0.503	0.52	0.010 / 0.505	0.37	0.014 / 0.507	0.26
ChaCha20	0.010 / 0.505	0.35	0.014 / 0.507	0.24	0.018 / 0.509	0.17
HC-128	0.012 / 0.506	0.30	0.016 / 0.508	0.20	0.020 / 0.510	0.15
BBS	0.022 / 0.512	0.10	0.040 / 0.525	0.01	0.070 / 0.545	0.001
MT19937	0.120 / 0.620	$< 10^{-6}$	0.220 / 0.690	$< 10^{-10}$	0.360 / 0.760	$< 10^{-12}$
GAN PRNG	0.060 / 0.545	0.002	0.120 / 0.600	$< 10^{-5}$	0.160 / 0.635	$< 10^{-6}$
LPN PRNG	0.016 / 0.508	0.22	0.024 / 0.512	0.11	0.032 / 0.516	0.05

Table 9
PNB Test Results.

	GAN-LPN PRNG	AES-CTR	ChaCha20	HC-128	BBS	MT	GAN PRNG	LPN PRNG
ONB	215	208	205	219	202	233	237	199

and short-range correlations), (ii) a Transformer encoder (sensitive to longer-range and global dependencies, subject to the max sequence length constraint), and (iii) an MLP baseline (requires more data to exploit high-dimensional dependencies). All D' models are trained *independently* of the GAN training discriminator and are not fine-tuned from it.

Training protocol. We use Adam and `BCEWithLogitsLoss` with early stopping (patience on validation loss). Batch size is chosen adaptively based on `effective_length` to fit the available memory. For longer lengths, we reduce the maximum number of epochs to control runtime while keeping the same early-stopping criterion.

Evaluation metrics. On the held-out test set, we report accuracy, AUC, and the standard distinguishing advantage

$$\widehat{\text{Adv}} = \left|2 \cdot \widehat{\text{Acc}} - 1\right|.$$

We additionally report confusion matrices, bootstrap confidence intervals (1000 resamples), and exact binomial test p-values for whether $\widehat{\text{Acc}} > 0.5$.

Table 8 shows that AES-CTR, ChaCha20, HC-128, and GAN-LPN remain indistinguishable from random under the retrained discriminator D', whereas MT19937 and the GAN-only baseline exhibit detectable weaknesses. This supports the statistical robustness of GAN-LPN and the necessity of combining generative models with the LPN-based construction.

5.6. Resistance to next-Bit prediction attacks: Practical next-Bit test

A fundamental threat model in the cryptanalysis of pseudorandom number generators is the Next-Bit Prediction Attack. In this adversarial scenario, an attacker who has observed the first k bits of a generated sequence attempts to predict the $(k+1)$-th bit with probability significantly greater than $1/2$. A PRNG that is secure against such an attack is said to pass the next-bit test, which is a cornerstone of cryptographic security definitions[41].

The Practical Next-Bit (PNB) Test, based on the Sadeghiyan-Mohajeri universal test framework[42], operationalizes this concept through an efficient pattern-based analysis. The core algorithm employs a binary tree structure where each node represents a binary pattern of length w, and systematically examines whether the occurrence frequencies of sibling patterns (differing only in the final bit) exhibit statistically significant deviations from randomness.

The test constructs a complete binary tree up to depth $w = \lfloor \log_2(L) \rfloor - 2$, where L is the sequence length. For each pair of sibling nodes representing patterns P_0 and P_1 (which are identical except for the final bit), the algorithm counts their occurrences C_0 and C_1 throughout the sequence. The predictability is assessed using a threshold derived from the chi-square distribution:

$$\text{threshold} = \frac{1 + \sqrt{\chi^2_{1,0.95}/L}}{2}$$

where $\chi^2_{1,0.95}$ is the 95th percentile of the chi-square distribution with 1 degree of freedom. A pattern pair is considered predictable if $C_0 + C_1 \geq 10$ and either $C_0/(C_0 + C_1)$ or $C_1/(C_0 + C_1)$ exceeds this threshold. The Observed Number of Predictable Bits (ONB) is the total count of such predictable pattern pairs detected across all examined layers.

To assess our proposed generator under this practical adversarial scenario, we conducted the PNB test on sequences generated by our GAN-LPN PRNG and seven established benchmarks. The test was configured with a sequence length of $L = 2^{17}$ bits and executed using parallel processing to ensure computational efficiency.

The results, summarized in Table 9, demonstrate that the proposed GAN-LPN PRNG exhibits a level of next-bit unpredictability comparable to widely used cryptographic generators. Specifically, the GAN-LPN PRNG achieves an ONB of 215, which is close to AES-CTR (208), ChaCha20 (205), and HC-128 (219), and within the same range as BBS (202). In contrast, MT and the baseline GAN PRNG yield higher ONB values (233 and 237, respectively) under the same configuration, indicating comparatively stronger detectable predictability in this practical test. Overall, the ONB of the proposed GAN-LPN PRNG remains in the low hundreds and aligns with the magnitude observed for established cryptographic constructions.

5.7. Performance and efficiency benchmarking

The computational efficiency of a pseudorandom number generator is a key factor for real-world deployment. In addition to the statistical tests reported in Section 5.4– 5.6, we benchmark the runtime performance of our GAN-LPN PRNG and several representative baselines. We focus on three practical measurements: *throughput*, *latency*, and *CPU utilization*. The results are summarized in Table 10 and Fig. 7.

Benchmark methodology. All algorithms are evaluated on the same hardware/software platform described in Section 5.2. For each method, we generate a fixed-length output per call (identical across algorithms) and repeat the experiment multiple times; we report average values after a small number of warm-up runs to exclude one-time initialization overhead. For GAN-based generators, we additionally evaluate batched generation (BatchSize = 1 vs. 100) because the same inference code path naturally supports vectorized inputs without any algorithmic modification.

Table 10
Efficiency comparison of PRNGs (throughput and latency).

	Throughput (MB/s)	Latency (ms)	Mode
GAN-LPN PRNG	0.04	737.64	CPU, BatchSize = 1
	0.10	533.55	CPU, BatchSize = 100
	0.11	3.81	GPU, BatchSize = 1
	4.32	106.71	GPU, BatchSize = 100
AES-CTR	1.32	0.04	CPU
ChaCha20	0.45	0.02	CPU
HC-128	1.06	0.01	CPU
BBS	0.03	0.60	CPU
MT19937	2.50	0.003	CPU
GAN PRNG	4.01	0.64	CPU, BatchSize = 1
	105.76	2.40	CPU, BatchSize = 100
	2.74	0.13	GPU, BatchSize = 1
	274.25	0.48	GPU, BatchSize = 100
LPN PRNG	0.01	15.24	CPU

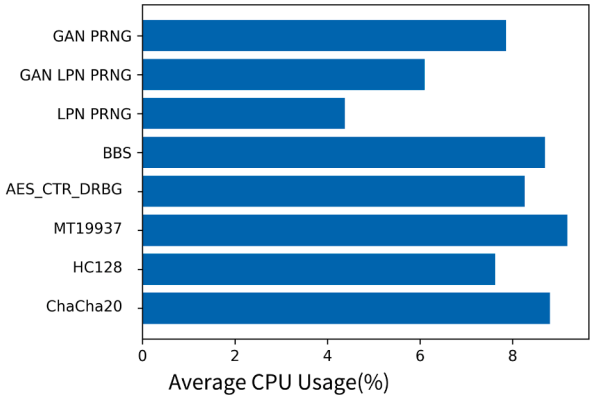

Fig. 7. Average CPU utilization (%) during benchmarking for different PRNGs.

- **Throughput (MB/s)** is computed as the total generated output size divided by wall-clock time. For BatchSize = B, a single call returns B independent output streams of the same fixed length; throughput is computed using the *total* output size produced by that call.
- **Latency (ms)** is the wall-clock time per generation call. For batched settings, this is the time to generate *all* B streams in the batch; the effective per-stream latency can be approximated as Latency/B.
- **CPU utilization (%)** is the average CPU usage observed during the benchmark window (as reported by the operating system). While CPU utilization is not a direct measurement of energy in Joules, it provides a reproducible indicator of computational load and helps characterize deployment cost on CPU-centric platforms.

Throughput and latency. Table 10 shows that the GAN-LPN PRNG has high latency in CPU single-instance mode (BatchSize= 1), which is expected because its core cost is dominated by neural-network inference. However, the implementation supports vectorized generation: multiple independent inputs can be stacked and processed in a single forward pass, enabling the underlying tensor kernels (matrix multiplications and element-wise activations) to reuse optimized parallel routines.

This behavior is reflected in the BatchSize comparison. On CPU, increasing BatchSize from 1 to 100 increases throughput from 0.04 to 0.10 MB/s (a 2.5× improvement). Although the *per-call* latency remains on the order of hundreds of milliseconds (from 737.64 to 533.55 ms), the effective per-stream latency decreases substantially (approximately 7.38 ms/stream to 5.34 ms/stream) because one call produces 100 independent streams.

The same effect is more pronounced on GPU. The GAN-LPN PRNG reaches 0.11 MB/s at BatchSize= 1 with 3.81 ms latency, while BatchSize= 100 increases throughput to 4.32 MB/s. Although the batch call latency (106.71 ms) is higher than the single-stream call, it produces 100 streams in that time, corresponding to an effective per-stream latency of about 1.07 ms. Overall, the GPU + batch configuration provides a 108× throughput improvement over CPU single-instance mode (4.32/0.04).

For classical baselines (ChaCha20, HC-128, MT19937, AES-CTR-DRBG), batching is not an intrinsic execution mode in typical reference implementations and would require interface/implementation adaptations that may not be comparable across libraries. Therefore, we report their standard single-stream CPU performance as commonly used in practice. As expected, these lightweight constructions exhibit very low latency (sub-millisecond) and stable throughput in CPU mode, whereas constructions such as BBS and the basic LPN PRNG are significantly slower due to heavy arithmetic or linear-algebraic costs.

CPU utilization. Fig. 7 reports the average CPU usage during the benchmarks. This complements throughput/latency by indicating the CPU-side load imposed by each generator under the same task. In particular, neural-network-based generators may show nontrivial CPU utilization even when GPU acceleration is used, due to framework/runtime overheads (e.g., data marshaling, kernel launches, and scheduling). Conversely, classical stream/DRBG baselines typically incur low overhead and exhibit utilization patterns dominated by tight, optimized CPU loops.

5.8. Comprehensive comparison with existing PRNGs

A holistic evaluation of pseudorandom number generators necessitates a multi-dimensional analysis encompassing security foundations, resilience against emerging threats, and practical performance. Table 11 presents a systematic comparison of representative PRNG schemes across key attributes. The results position the proposed GAN-LPN PRNG within the broader design landscape and highlight its distinct design trade-offs.

The security foundation of a PRNG delineates its trust model. Our proposed GAN-LPN PRNG is supported by a reduction to the underlying LPN problem. Under the assumption that the selected LPN parameters remain hard for QPT adversaries, this provides a conditional post-quantum interpretation. This distinguishes it from heuristic schemes such as AES-CTR and ChaCha20, whose practical security is not presented here through a comparable post-quantum hardness reduction,

Table 11
Comprehensive comparison with existing PRNGs.

Scheme	Security Foundation	PQ Interpretation	Forward Security	Throughput (MB/s)
GAN-LPN PRNG	Reduction to LPN	Conditional on QPT-hard LPN	Yes	4.32 (GPU, batched)
AES-CTR	Heuristic	No comparable PQ reduction reported here	Yes	1.32 (CPU)
ChaCha20	Heuristic	No comparable PQ reduction reported here	Yes	0.45 (CPU)
HC-128	Heuristic	No comparable PQ reduction reported here	Yes	1.06 (CPU)
BBS	Reduction to factoring	Not PQ-secure	Yes	0.03 (CPU)
MT	Heuristic	N/A	No	2.50 (CPU)
GAN PRNG	Heuristic	Not based on a hardness assumption	Yes	274.25 (GPU)
LPN PRNG	Reduction to LPN	Conditional on QPT-hard LPN	Yes	0.01 (CPU)

and from BBS, whose classical security foundation does not provide post-quantum protection against Shor-type attacks.

Regarding practical performance, the throughput data confirms the expected trade-offs. The GAN-LPN PRNG, when leveraging GPU acceleration, achieves a throughput of 4.32 MB/s. While this figure is obtained in a GPU-batched setting and is therefore not directly identical to the CPU-based baselines in Table 11, it indicates that parallel evaluation can partially offset the computational overhead introduced by the learned sampler.

Overall, the comparative analysis shows that the GAN-LPN PRNG occupies a distinct point in the design space. It combines reduction-based security analysis, a conditional post-quantum interpretation under the assumed hardness of the underlying LPN instance, and moderate throughput in the GPU-batched setting.

6. Conclusion

This study explores the possibility of integrating GANs with post-quantum cryptographic assumptions. It proposes a GAN-based noise sampler and a corresponding variant of the LPN problem, and based on these, constructs a PRNG within an LPN-based framework. By embedding the distribution learning capability of neural networks into the framework of cryptographic hard problems, this study provides a preliminary scheme for investigating how generative models may be incorporated into cryptographic design under explicit security assumptions and bounded deviation terms. Experiments show that the proposed scheme can generate random sequences that pass standard statistical tests, and that it performs favorably in Hamming distance analysis and a practical next-bit prediction test. These results provide empirical support for the statistical quality of the generated outputs, complementing the reduction-based security analysis developed in the paper.

However, the framework faces computational efficiency challenges due to the high-dimensional noise modeling inherent in deep neural networks and the adversarial training dynamics of WGANs, resulting in significant overhead that may limit deployment in high-throughput encryption scenarios. To address this, future research will focus on two directions: (1) algorithmic optimization through hybrid architectures (e.g., Transformer-GAN integration) or knowledge distillation techniques to reduce inference complexity via model compression; and (2) hardware acceleration strategies leveraging parallel architectures and pipeline designs for more efficient sampling. These directions may improve throughput while preserving the intended security properties under the stated assumptions, thereby supporting broader practical exploration of learned-sampler-based PRNG constructions in domains such as IoT authentication, Byzantine fault-tolerant distributed systems, and privacy-preserving machine learning.

CRediT authorship contribution statement

Xuguang Wu: Writing – original draft, Methodology, Conceptualization; **Yiliang Han:** Writing – original draft, Supervision, Resources; **Minqing Zhang:** Supervision, Resources; **Shuaishuai Zhu:** Writing – original draft, Visualization, Data curation; **Xu An Wang:** Formal analysis, Visualization.

Data availability

Code & Data: Publicly available at https://gitee.com/guangandy/ganlpnprng.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work is supported Natural Science Basic Research Plan in Shaanxi Province of China (No. 2025JC-YBMS-664).

References

[1] T. van der Zant, M. Kouw, L. Schomaker, Generative artificial intelligence, Springer, 2013.
[2] C. Yinka-Banjo, O.-A. Ugot, A review of generative adversarial networks and its application in cybersecurity, Artif. Intell. Rev. 53 (2020) 1721–1736.
[3] J. Katz, Y. Lindell, Introduction to Modern Cryptography: Principles and Protocols, Chapman and Hall/CRC, New York, New York, 2007. https://doi.org/10.1201/9781420010756
[4] M. Rosulek, The joy of cryptography, 2021, ([Online]). https://joyofcryptography.com/.
[5] I.J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative Adversarial Networks, 2014, arXiv:1406.2661https://doi.org/10.48550/arXiv.1406.2661
[6] M. De Bernardi, M.H.R. Khouzani, P. Malacaria, Pseudo-Random number generation using generative adversarial networks, in: ECML PKDD 2018 Workshops, 11329, Springer International Publishing, Cham, 2019, pp. 191–200. https://doi.org/10.1007/978-3-030-13453-2_15
[7] R. Oak, C. Rahalkar, D. Gujar, Poster: using generative adversarial networks for secure pseudorandom number generation, in: Proceedings of the 2019ACM SIGSAC Conference on Computer and Communications Security, ACM, London United Kingdom, 2019, pp. 2597–2599. https://doi.org/10.1145/3319535.3363265
[8] H. Kim, Y. Kwon, M. Sim, S. Lim, H. Seo, Generative adversarial networks-Based pseudo-Random number generator for embedded processors, in: Information Security and Cryptology – ICISC 2020, Springer, Cham, 2021, pp. 215–234. https://doi.org/10.1007/978-3-030-68890-5_12
[9] K. Okada, K. Endo, K. Yasuoka, S. Kurabayashi, Learned pseudo-Random number generator: WGAN-GP for generating statistically robust random numbers, PLoS ONE 18 (6) (2023) e0287025. https://doi.org/10.1371/journal.pone.0287025
[10] X. Wu, Y. Han, M. Zhang, Y. Li, S. Cui, GAN-Based pseudo random number generation optimized through genetic algorithms, Complex Intell. Syst. 11 (1) (2024) 31. https://doi.org/10.1007/s40747-024-01606-w
[11] M. Arjovsky, S. Chintala, L. Bottou, Wasserstein generative adversarial networks, in: International Conference on Machine Learning, PMLR, 2017, pp. 214–223.
[12] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, A.C. Courville, Improved training of wasserstein gans, Adv. Neural Inf. Process. Syst. 30 (2017).
[13] T. Chakraborty, R.K.S. Ujjwal, S.M. Naik, M. Panja, B. Manvitha, Ten years of generative adversarial nets (GANs): a survey of the state-of-the-Art, Mach. Learn.: Sci. Technol. 5 (1) (2024) 011001. https://doi.org/10.1088/2632-2153/ad1f77
[14] S. Ayyaz, S.M. Malik, A comprehensive study of generative adversarial networks (GAN) and generative pre-Trained transformers (GPT) in cybersecurity, in: 2024 Sixth International Conference on Intelligent Computing in Data Sciences (ICDS), 2024, pp. 1–8. https://doi.org/10.1109/ICDS62089.2024.10756505
[15] H. Zhang, A theoretical study of improving the training stability of GAN based on wasserstein distance as optimization objective, Adv. Appl. Math. 14 (2025) 601. https://doi.org/10.12677/aam.2025.145286
[16] E. Barker, J. Kelsey, Recommendation for Random Number Generation Using Deterministic Random Bit Generators, NIST Special Publication 800-90A Rev. 1, National Institute of Standards and Technology, 2015. Revision 1, https://doi.org/10.6028/NIST.SP.800-90Ar1
[17] M.S. Turan, E. Barker, J. Kelsey, K.A. McKay, M.L. Baish, M. Boyle, Recommendation for the Entropy Sources Used for Random Bit Generation, NIST Special Publication 800-90B, National Institute of Standards and Technology, 2018. https://doi.org/10.6028/NIST.SP.800-90B
[18] N. Standards, Recommendation for Random Bit Generator Constructions, NIST Special Publication 800-90C, National Institute of Standards and Technology, 2025. Final version, https://doi.org/10.6028/NIST.SP.800-90C
[19] L. Blum, M. Blum, M. Shub, A simple unpredictable pseudo-Random number generator, SIAM J. Comput. 15 (2) (1986) 364–383. https://doi.org/10.1137/0215025
[20] M. Matsumoto, T. Nishimura, Mersenne twister: a 623-Dimensionally equidistributed uniform pseudo-Random number generator, ACM Trans. Model. Comput. Simul. 8 (1) (1998) 3–30. https://doi.org/10.1145/272991.272995
[21] H. Wu, The stream cipher HC-128, in: New Stream Cipher Designs, Springer, Berlin, Heidelberg, 2008, pp. 39–47. https://doi.org/10.1007/978-3-540-68351-3_4
[22] D.J. Bernstein, et al., Chacha, a variant of salsa20, in: Workshop Record of SASC, Lausanne, Switzerland, 2008, pp. 3–5.
[23] L.E. Bassham, III, A.L. Rukhin, J. Soto, J.R. Nechvatal, M.E. Smid, E.B. Barker, S.D. Leigh, M. Levenson, M. Vangel, D.L. Banks, et al., Sp 800-22 rev. 1a. a statistical test suite for random and pseudorandom number generators for cryptographic applications, 2010.
[24] S.C.A. China, Randomness Test Specification, Cryptography Industry Standard of the P.R. China GM/T 0005–2021, Technical Standard, China Standard Press, 2021. [Online]. https://www.chinesestandard.net/pdf/BOOK.aspx/GMT0005-2021.
[25] X. Chen, W. Shu, Z. Zhou, Algorithms for sparse LPN and LSPN against low-Noise (extended abstract), in: Proceedings of Thirty Eighth Conference on Learning Theory, PMLR, 2025, pp. 1091–1093.
[26] Q. Dao, A. Jain, Lossy cryptography from code-Based assumptions dense-Sparse LPN: a new subexponentially hard LPN variant in SZK, J. Cryptol. 38 (4) (2025) 32. https://doi.org/10.1007/s00145-025-09553-6

[27] S. Ragavan, N. Vafa, V. Vaikuntanathan, Indistinguishability obfuscation from bilinear maps and LPN variants, in: Theory of Cryptography Conference, Springer, 2025, pp. 3–36.
[28] H. Liu, X. Wang, K. Yang, Y. Yu, The hardness of LPN over any integer ring and field for PCG applications, in: M. Joye, G. Leander (Eds.), Advances in Cryptology – EUROCRYPT 2024, Springer Nature Switzerland, Cham, 2024, pp. 149–179. https://doi.org/10.1007/978-3-031-58751-1_6
[29] D. Abram, G. Malavolta, L. Roy, Trapdoor hash functions and PIR from low-Noise LPN, Cryptol. ePrint Arch. (2025).
[30] C.M. Bishop, Pattern recognition and machine learning, Springer Google Schola 2 (2006) 1122–1128.
[31] F. Horger, T. Würfl, V. Christlein, A. Maier, Towards arbitrary noise augmentation-deep learning for sampling from arbitrary probability distributions, in: Machine Learning for Medical Image Reconstruction: First International Workshop, MLMIR 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16, 2018, Proceedings 1, Springer, 2018, pp. 129–137.
[32] D. Perekrestenko, S. Müller, H. Bölcskei, Constructive universal high-dimensional distribution generation through deep relu networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 7610–7619.
[33] M. Mukherjee, A. Tchamkerten, M. Yousefi, Approximating probability distributions by relu networks, in: 2020IEEE Information Theory Workshop (ITW), IEEE, 2021, pp. 1–5.
[34] Y. Yang, Z. Li, Y. Wang, On the capacity of deep generative networks for approximating distributions, Neural Netw. 145 (2022) 144–154.
[35] D. Yarotsky, Learning high-dimensional targets by two-parameter models and gradient flow, arXiv preprint arXiv:2402.17089 (2024).
[36] B. Applebaum, Y. Ishai, E. Kushilevitz, On pseudorandom generators with linear stretch in NC0, Comput. Complex. 17 (1) (2008) 38–69. https://doi.org/10.1007/s00037-007-0237-6
[37] S. Bogos, D. Korolija, T. Locher, S. Vaudenay, Towards efficient LPN-based symmetric encryption, in: International Conference on Applied Cryptography and Network Security, Springer, 2021, pp. 208–230.
[38] D. Yarotsky, Error bounds for approximations with deep reLU networks, Neural Netw. 94 (2017-10-01) 103–114. https://doi.org/10.1016/j.neunet.2017.07.002
[39] D. Johnston, Random Number Generators Principles and Practice: A Guide for Engineers and Programmers, De|G Press, Berlin, DEU, Berlin, DEU, 2018.
[40] Y. Han, H. Feng, X. Wu, Y. Sun, Y. Wang, A survey of neural network-Based evaluation of random number generators, Netinfo Secur. 26 (2) (2026) 171. https://doi.org/10.3969/j.issn.1671-1122.2026.02.001
[41] A. Lavasani, T. Eghlidos, Practical next bit test for evaluating pseudorandom sequences, Scientia Iranica 16 (1) (2009).
[42] B. Sadeghiyan, J. Mohajeri, A new universal test for bit strings, in: J. Pieprzyk, J. Seberry (Eds.), Information Security and Privacy, Springer, Berlin, Heidelberg, 1996, pp. 311–319. https://doi.org/10.1007/BFb0023309